FDRM VIX Project: Study Guide

Oct 6, 2026 · @Subham Agrawal

The project in one page

The project asks one practical question. Can a long-term Nifty investor buy crash protection cheaply by timing it with India VIX? The answer from 2020 to 2026 data is yes, but only as rare-event insurance. This guide explains every step behind that answer so you can defend it and extend it.

The assignment had four parts, and each maps to one piece of the work:

Assignment asks	What we did	Where it lives
Implied vs realised vol, the volatility risk premium	Compared India VIX with the volatility Nifty actually delivered over the next 21 trading days	Report section 3, code section 3, slide 4
India VIX construction and use as a sentiment gauge	Explained NSE's formula, re-implemented it on a synthetic option chain, measured how VIX reacts to Nifty moves	Report section 4, code section 2, slides 5 and 6
VIX regimes vs Nifty returns	Split days into Low (VIX < 13), Normal (13 to 20) and High (> 20), then tested returns and volatility in each	Report section 5, code section 4, slide 7
Option premiums around 4 June 2024; a hedge bought in low VIX and monetised in spikes; a backtest across regimes	Event study of the election, a rule-based put strategy and a day-by-day backtest against six alternatives	Report sections 6 to 9, code sections 5 to 7, slides 8 to 16

How the pieces fit, in the order the code runs:

1. Load two real daily price files: Nifty 50 and India VIX. Join them on date.
2. Compute returns and realised volatility from Nifty, and label each day with its VIX regime.
3. Measure the volatility risk premium and the regime statistics.

4. Price options with Black-Scholes, using India VIX as the volatility, to study the election and to run the backtest.
5. Simulate seven strategies day by day from January 2020 to April 2026. Record every option trade.
6. Stress-test the result by changing thresholds, pricing assumptions, tenor and the sell rule.
7. Save every number to `output/results.json` and every chart to `figures/`. The report and the deck are written from those files.

The five facts to remember:

- India VIX was 2.2 vol points above the volatility that followed, on 78% of days. Options are usually overpriced.
- The premium was largest (4.0 points) right after shocks, so that is the time to sell protection.
- Nifty was three times as volatile in high-VIX regimes (30%) as in low ones (9%), but average returns did not differ significantly.
- Before the 2024 election, option prices doubled over six weeks. On result day Nifty fell 5.9%, which was almost exactly the move VIX had priced in, and VIX collapsed 29% the next day.
- The proposed hedge (a 3-month put bought only when VIX < 13, sold into spikes) cut the worst fall from 38% to 22% and returned 13.9% a year against 11.1% unhedged. Almost all of that came from the 2020 COVID crash. Without it, the hedge cost about 0.5% a year.

Files you have (all in the shared `fdrm-vix` folder):

File	What it is
`fdrm_vix_analysis.py` and `.ipynb`	The analysis, as a script and as an already-run notebook. Explained in the Code walkthrough tab.
`data/`	The two raw CSV files
`output/`	Every table as a CSV, plus `results.json` with every number quoted anywhere
`figures/`	The 8 charts
`fdrm_vix_presentation.pptx` and `.pdf`	The 17-slide deck. Explained in the Deck walkthrough tab.
`build_deck.js`	The code that generates the deck

File	What it is
The report doc	Nifty Options, India VIX and a VIX-Timed Protection Strategy

Theory 1: implied vs realised volatility, and the volatility risk premium

Volatility is how much a price jumps around. It is quoted as an annualised standard deviation of returns, in percent. A volatility of 16% means a typical one-year move of about 16%, a typical one-month move of about 16% ÷ √12 ≈ 4.6%, and a typical one-day move of about 16% ÷ √252 ≈ 1%.

Realised (historical) volatility is measured from past prices. Take daily log returns r = ln(P_t / P_{t-1}). Compute their standard deviation over a window, here 21 trading days (about one month). Multiply by √252 to annualise, because there are about 252 trading days a year and variance grows in proportion to time.

```
RV = √252 × sd(r_t+1, …, r_t+21) × 100
```

Implied volatility is read from option prices. An option's price depends on how much the market expects the underlying to move. Run the Black-Scholes formula backwards: given the option's price, find the volatility that produces it. That number is the implied volatility (IV). It is the market's forecast of future volatility, plus whatever extra people are willing to pay for protection.

The volatility risk premium (VRP) is the gap between the two. In this project:

```
VRP(t) = India VIX(t) − RV over the next 21 trading days
```

The timing matters. VIX on day t is compared with the volatility that happened over the next 21 trading days, because VIX is a forecast of the next 30 calendar days. Comparing VIX with past volatility would only measure how VIX follows the market, not whether it forecast well.

Why is the VRP usually positive? Option sellers are selling insurance. Insurance companies charge more than the expected payout because they bear the risk of a rare catastrophic loss. Option sellers do the same. Investors who own stocks will pay above fair value for puts that protect them in a crash, because crashes hurt most exactly when they can least afford it. So implied volatility sits above realised volatility on most days. On the rare days a crash arrives, realised volatility shoots above it.

What we found for Nifty, 2020 to 2026:

- India VIX averaged 17.3, while realised volatility over the next 21 days averaged 15.1. The average VRP was +2.2 vol points.
- VIX was above subsequent realised volatility on 78% of days.
- The Newey-West t-statistic for the mean VRP was 2.8, so the premium is statistically significant (see the statistics section for why Newey-West).
- The VRP was largest in the high-VIX regime (4.0 points). After a shock, fear stays high while actual volatility is already calming down.
- VIX correlates 0.86 with past 21-day volatility but only 0.61 with future 21-day volatility. VIX reacts to what has just happened more than it predicts what comes next.

Why this matters for the strategy. If options are usually overpriced, a hedger who buys them all the time loses money on average. The edge is in when you buy. Options are cheapest in absolute terms when VIX is low, and most overpriced just after a spike, which is the best time to sell. The whole strategy follows from that.

Theory 2: India VIX, how it is built and how to read it

India VIX is NSE's measure of the volatility the options market expects for Nifty over the next 30 calendar days, in annualised percent. NSE launched it in 2008, adapting the CBOE method behind the US VIX. It is calculated in real time from the order book of Nifty options. You can think of it as the implied volatility of a 30-day Nifty option, averaged across many strikes instead of taken from one.

How to read a VIX number. Divide by $\sqrt{12}$ (about 3.5) for a one-month move, or by about 16 ($\sqrt{252} \approx 15.9$) for a one-day move. These are one-standard-deviation moves, so the market expects a move that size or smaller about two-thirds of the time.

India VIX	Expected 1-day move	Expected 1-month move	Regime in this project
11	±0.7%	±3.2%	Low
16	±1.0%	±4.6%	Normal
25	±1.6%	±7.2%	High
60	±3.8%	±17.3%	High (March 2020)

How NSE builds it, step by step:

1. Take Nifty options for two expiries: the near month and the next month.
2. Find the forward level F of Nifty from put-call parity, at the strike where call and put prices are closest. Let K0 be the first strike at or below F.

3. Use only out-of-the-money options: puts with strikes below K0 and calls with strikes above it. At K0, average the put and call. Q(K) is the mid-point of each option's best bid and ask.
4. For each expiry, compute the variance with the formula below. T is the time to expiry in years (NSE measures it in minutes), R is a short-term interest rate, and ΔK is the gap between neighbouring strikes.
5. Interpolate between the two expiries' variances to get a constant 30-day variance.
6. India VIX = 100 × the square root of that 30-day variance.

$$\sigma^2 = (2/T) \cdot \Sigma_i \, (\Delta K_i / K_i^2) \cdot e^{RT} \cdot Q(K_i) - (1/T) \cdot (F/K_0 - 1)^2$$

Why the formula looks like this. It comes from a result in finance: the fair value of future variance equals a weighted portfolio of out-of-the-money options, each weighted by $1/K^2$. You don't need a pricing model to compute it; that is why it is called model-free. The second term is a small correction for K0 not being exactly at the forward. The $1/K^2$ weight matters: low-strike puts get more weight than high-strike calls. When investors pay up for crash protection (put skew), VIX rises even if at-the-money volatility does not.

Our check of the formula. The script prices a synthetic Nifty option chain with Black-Scholes and pushes it through the formula. With every option at 15% volatility, it returns exactly 15.0, so the formula is implemented correctly. With 15% at the money plus put skew, it returns 16.1. So VIX is part volatility forecast and part price of crash insurance.

India VIX as a sentiment indicator. It is called the fear index because it rises when investors rush to buy protection. Four properties show up in the data:

- **Inverse to the market.** VIX rose 2.3% on the average down day and fell 1.9% on the average up day. The correlation between daily Nifty returns and VIX changes was −0.56. This asymmetry is called the leverage effect: falling prices raise risk more than rising prices lower it.
- **Clusters.** Calm and fearful periods each last weeks or months, which is what makes regimes useful.
- **Mean-reverts.** Every spike in the sample decayed back towards 12 to 16. A very high VIX has historically been followed by recovery, not further panic.
- **Prices known events in advance.** Before scheduled events (elections, budgets, central bank decisions), VIX climbs even when Nifty is flat, then collapses once the news is out.

What VIX does not tell you. It does not say which way the market will move. A high VIX means a big move is expected, up or down. It is also not directly tradeable for most investors. NSE has listed India VIX futures since 2014, but they trade thinly, so in practice VIX is a signal and Nifty options are the instrument.

Theory 3: option pricing ideas used

Puts, calls and a protective put. A Nifty put with strike K gives the right to sell Nifty at K on expiry. Its payoff is max(K − S, 0), where S is Nifty at expiry. A call pays max(S − K, 0). Nifty options on NSE are European (exercised only at expiry) and cash-settled. Owning the portfolio plus a put creates a floor: below K, every point the portfolio loses, the put gains. A put with K 5% below spot is called 5% out of the money (OTM), or a "95% put". It is cheaper than an at-the-money (ATM) put because the first 5% of any fall is not covered, just like an insurance deductible.

Black-Scholes-Merton (BSM). The script prices every option with this formula, extended for a dividend yield q:

$$P = K \cdot e^{-rT} \cdot N(-d_2) - S \cdot e^{-qT} \cdot N(-d_1)$$
$$d_1 = \left[\ln(S/K) + (r - q + \sigma^2/2) \cdot T \right] / (\sigma \cdot \sqrt{T})$$
$$d_2 = d_1 - \sigma \cdot \sqrt{T}$$

Input	Meaning	Value used
S	Nifty level that day	Daily close from the data
K	Strike	95% of spot, rounded to the nearest 50 (NSE strikes are 50 apart)
T	Time to expiry in years	Calendar days ÷ 365
r	Risk-free rate	6.5%, roughly the 91-day Treasury bill yield
q	Dividend yield of Nifty	1.3%
σ	Implied volatility	India VIX plus a skew add-on (below)
N(·)	Standard normal cumulative probability	From `scipy.stats.norm`

The call price comes from **put-call parity**, which holds for any European options without needing a model: C − P = S·e^(−qT) − K·e^(−rT). That is exactly the line `bs_call = bs_put + S·e^(−qT) − K·e^(−rT)` in the code.

Skew: why the volatility input is not just VIX. If Black-Scholes were literally true, every strike would trade at the same implied volatility. In reality, OTM puts trade at higher implied volatility than ATM options because everyone wants crash protection and few want to sell it. Plotted against strike this looks like a smirk, falling from low strikes to high. The project models it in one line: put IV = VIX + 0.6 vol points for every 1% the strike is

below spot. A 5% OTM put therefore gets VIX + 3 points. On top of that, buying costs 0.5 vol points more and selling gets 0.5 less, to mimic the bid/ask spread. At VIX 12, the hedge put is bought at 12 + 3 + 0.5 = 15.5% implied volatility.

What the hedge put costs. Worked example with Nifty at 24,000 and a 22,800 put (5% OTM), using the project's settings:

India VIX	1-month put	3-month put
12 (Low)	52 points, 0.22% of spot	204 points, 0.85% of spot
16 (Normal)	110 points, 0.46%	345 points, 1.44%
25 (High)	281 points, 1.17%	702 points, 2.92%

The same protection costs about 3.5 to 5 times as much at VIX 25 as at VIX 12. That is the first reason to buy only in low regimes.

The Greeks you need. These are the sensitivities of the option price. The numbers are for the 3-month 22,800 put at VIX 12.

- **Delta (−0.19).** The put gains about 0.19 points for each 1-point fall in Nifty. Delta grows towards −1 as Nifty falls through the strike, so protection gets stronger the further the market falls.
- **Gamma.** How fast delta changes. Long puts have positive gamma: they gain more and more per point in a crash. This curvature is called convexity.
- **Vega (+33 points per vol point).** If VIX rises from 12 to 25 with Nifty unchanged, this put gains roughly 33 × 13 ≈ 430 points, more than doubling. In a crash, Nifty falls and VIX jumps together, so the put gains from delta and vega at once. Selling the put in the spike captures the vega gain, which disappears again once VIX falls back.
- **Theta (−2.1 points a day).** The option loses value every day just from time passing. The 1-month put loses 2.8 points a day while costing a quarter as much, so short-dated puts decay much faster per rupee of premium. This is the second reason for the 3-month tenor: rolling it monthly sells it before the steep decay of its final weeks.

The ATM straddle as a measure of expected move. A straddle is an ATM call plus an ATM put. It pays off when Nifty moves far in either direction, so its price is the market's estimate of the size of the move. A handy approximation:

$$\text{Straddle} \approx \sqrt{2/\pi} \cdot \sigma \cdot \sqrt{T} \cdot S \approx 0.8 \cdot \sigma \cdot \sqrt{T} \cdot S$$

At VIX 14 a 30-day straddle costs 0.8 × 0.14 × √(30/365) ≈ 3.2% of spot. At VIX 24.6 (31 May 2024) it costs 5.6%. The event study uses the straddle in percent of spot to compare option prices across dates when Nifty was at different levels.

Term structure, and the biggest pricing caveat. India VIX measures 30-day volatility. Options with other tenors have their own implied volatility. In calm markets the curve usually slopes up: 3-month IV sits above 1-month IV. In a crisis it inverts, with 1-month IV far above 3-month IV, because traders expect the panic to fade. The project prices 3-month options with 30-day VIX. That probably makes the hedge look too cheap to buy in calm markets, and too valuable to sell in a spike. The skew and spread stress tests cover part of this, not all of it. Fixing it needs real option prices (see the data section and the ideas section).

Theory 4: the statistics used, and why each one

Log returns. The daily return is r = ln(P_t / P_{t−1}). Log returns add up over time (a week's log return is the sum of its daily ones), which makes rolling volatility and forward sums clean. For small moves they are almost equal to percentage returns. To report a percentage move, the code converts back with e^r − 1, which is why the election-day fall shows as −5.93%. Forward 21-day and 63-day returns are computed directly as simple returns, Close in 21 days ÷ Close today − 1.

Summary statistics in the regime tables.

- **Mean and median.** Average and middle value. When they differ a lot, the distribution is skewed by a few extreme days.
- **Standard deviation (sd).** The typical size of a move. Multiplied by √252 for a daily series, it becomes annualised volatility.
- **Hit rate.** The share of observations with a positive return.
- **5th percentile (p5).** The return that was beaten 95% of the time, a simple measure of a bad outcome. It is close to what risk managers call value at risk.

t-statistics and p-values. A t-statistic is an estimate divided by its standard error. Roughly, |t| above 2 means the estimate is unlikely to be pure noise (about 5% significance). A p-value is the probability of seeing a result at least this extreme if the true effect were zero. Small p means evidence against "no effect".

Newey-West standard errors, and why they are needed here. The forward 21-day return on Monday and on Tuesday share 20 of their 21 days. So 408 low-VIX days do not give 408 independent 21-day returns; they give closer to 408 ÷ 21 ≈ 19. The VRP has the same problem, because it uses forward realised volatility. An ordinary standard error treats every row as independent and makes results look far more certain than they are. The Newey-West (HAC, heteroskedasticity and autocorrelation consistent) estimator widens the standard error to allow for that overlap. The code uses a neat trick: regress the series on a constant. The coefficient is then just the mean, and asking for HAC errors with maxlags equal to the overlap gives the Newey-West t-statistic for that mean.

```
sm.OLS(x, np.ones(len(x))).fit(cov_type="HAC", cov_kwds={"maxlags": 21})
```

The lag is 1 for daily returns, 21 for 21-day returns and 63 for 63-day returns. With this correction, the mean VRP has t = 2.8, so it is genuinely significant. The +3.4% forward 21-day return after high-VIX days has t = 3.8.

Kruskal-Wallis test. Asks whether several groups come from the same distribution, using ranks instead of raw values. Ranks make it robust to fat tails, which matter for stock returns (one −13% day would dominate a test based on means). For same-day returns across the three regimes p = 0.66, so there is no evidence that low-, normal- and high-VIX days have different typical returns.

Levene's test. Asks whether several groups have the same variance. For daily returns by regime p < 0.001, so volatility clearly differs: 9% annualised in Low, 13% in Normal and 30% in High. VIX regimes sort risk, not return.

The script also runs both tests on forward 21-day returns, and both give p ≈ 0. Those two p-values ignore the overlap problem, so they overstate the evidence. The guide and report lean on the daily-return tests and the Newey-West t-statistics instead.

Correlation and the VIX beta. The correlation between daily Nifty log returns and daily changes in log VIX is −0.56. Regressing the VIX change on the Nifty return gives a slope of −2.7: a 1% fall in Nifty comes with about a 2.7% rise in VIX on average. The script also averages VIX changes separately on up and down days (−1.9% and +2.3%) to show the asymmetry.

Mincer-Zarnowitz regression. The standard test of whether a forecast is unbiased: regress the outcome on the forecast.

```
RV(next 21 days) = a + b · VIX(t) + error
```

A perfect, unbiased forecast would give a = 0 and b = 1. The result was a = 1.05, b = 0.81, R^2 = 0.38. At the average VIX of 17.3, the predicted realised volatility is 1.05 + 0.81 × 17.3 = 15.1, two points lower. VIX contains real information (it explains 38% of the variation in future volatility) but is biased upward. That bias is the VRP.

Predictive regression for returns. Regressing the next 21-day return on today's VIX gives +0.18% per VIX point (Newey-West t = 4.3). Read it carefully. Much of it comes from 2020, when VIX was extreme and Nifty then recovered sharply. It says high fear was overpriced in this sample. It is not a reliable timing rule.

Two limits to keep in mind. First, six years contain only three or four real stress episodes, so any statistic driven by crashes rests on very few independent events. Second, the regime cut-offs (13 and 20) and the strategy thresholds were chosen by looking at this

same data. That is in-sample choice, sometimes called data snooping. The sensitivity grids exist to show the conclusions do not depend on the exact numbers, but only a new sample can truly confirm them.

The data: where it came from, how it was checked and cleaned

What the project needed. Two daily time series: the Nifty 50 index (close, plus the intraday low for election day) and India VIX (close). Ideally it would also have had historical prices of actual Nifty option contracts. Those were not reachable, which is why option premiums are modelled (see below).

Where it came from. Both series are Yahoo Finance data. The first-choice sources (Yahoo Finance, NSE's website and Kaggle) all refused connections from the cloud machine the analysis ran on, with HTTP 403 errors. Public GitHub repositories that store copies of the same Yahoo data could be downloaded with `git clone`, so the files came from there:

File	Content	Original series	Downloaded from
`data/nifty50_daily_raw.csv`	4,554 days, 17 Sep 2007 to 13 Apr 2026. Columns Date, Adj Close, Close, High, Low, Open, Volume	Yahoo `^NSEI`	kalilurrahman/NIFTY_50_STOCK_DATA, file `NIFTY50_stock_history.csv`
`data/india_vix_daily_raw.csv`	1,627 days, 2 Jan 2020 to 24 Aug 2026. Columns "Sum of Open", "Sum of Close" and so on	Yahoo `^INDIAVIX`	tanya459/india-vix-market-data-analysis, file `Dataset/IndiaVIX_Cleaned_Dataset.csv`

If an examiner asks why GitHub: the numbers are Yahoo's, and they were checked against NSE's published closes (next paragraph). The mirror was only the delivery route.

Checks done before using the data.

- **Spot check against NSE.** On 4 June 2024 the files show Nifty closing at 21,884.50 and India VIX at 26.75. Both match NSE's published closes.
- **Known extremes.** India VIX's highest close in the file is 83.61 on 24 March 2020, the well-known COVID record. Its lowest is 9.15 on 26 December 2025.
- **Duplicates and gaps.** Neither file has duplicate dates. The Nifty file has no blanks. The VIX file has 3 blanks, all in derived columns the project does not use.
- **Adjusted close.** For the Nifty file, Adj Close equals Close on every row, as expected for an index (indices are not adjusted for dividends or splits).
- **Calendar mismatch.** Between 2 Jan 2020 and 13 Apr 2026, Nifty has 1,552 trading days and the VIX file 1,536. The 16 missing VIX days include 1 January in several years and the Saturday budget session of 1 February 2025. The inner join drops them.

Cleaning steps, in the order the code runs.

1. Read the Nifty CSV, parse dates, keep only Open, High, Low and Close as numbers, drop any duplicate dates and sort by date.
2. Read the VIX CSV with `encoding="utf-8-sig"`. The file starts with an invisible byte-order mark (BOM). Without this setting, the first column is named "Date" instead of "Date" and the date parsing fails.
3. Rename the Power BI-style column names ("Sum of Close" becomes VIX, and so on). Keep the four price columns.
4. Compute returns and realised volatility on the full Nifty history before joining, so a missing VIX day never distorts a Nifty return.
5. Inner-join the two on date and drop rows missing either close.

The result is 1,536 trading days from 2 January 2020 to 13 April 2026. It ends in April 2026 because the Nifty file does. It starts in 2020 because no India VIX history before 2020 was reachable.

Why option prices are modelled, and what that means. NSE publishes every option contract's daily prices in its F&O bhavcopy files, but those were blocked too. So each option is priced with Black-Scholes using India VIX plus skew as its volatility. Results therefore show what options would have cost at VIX-consistent prices, not what they actually traded at. Every report number that depends on this assumption is labelled "modelled".

How to get the data yourself (useful for extending the project on your own laptop, where these sites are not blocked):

Need	Source	How
Nifty and India VIX daily, quickly	Yahoo Finance	`pip install yfinance`, then `yf.download(["^NSEI", "^INDIAVIX"], start="2008-01-01")`. Check for gaps; Yahoo's India VIX history has holes.
Official index history	NSE	nseindia.com, Market Data, Historical Data: Indices, and the India VIX historical data page. Downloads as CSV, in date ranges of up to about a year.
Real option prices	NSE F&O bhavcopy	One file per trading day with every Nifty option's open, high, low, close, settlement price and open interest. Available from NSE's archives. Community Python packages such as `jugaad-data` can download them, though they break when NSE changes its website.
Dividends included	Nifty 50 Total Returns Index (TRI)	niftyindices.com, historical data. Use it to fix the "price index only" limitation.
Risk-free rate	RBI	91-day Treasury bill yields from the RBI's Database on Indian Economy (DBIE).

References for the theory and the index.

- NSE, *India VIX* white paper (computation methodology), available on nseindia.com under India VIX.
- Cboe, *VIX White Paper* (the model-free formula NSE adapted), available on cboe.com.
- Black, F. and Scholes, M. (1973), "The Pricing of Options and Corporate Liabilities", *Journal of Political Economy*. Merton, R. (1973), "Theory of Rational Option Pricing", for the dividend-yield version.
- Newey, W. and West, K. (1987), "A Simple, Positive Semi-Definite, Heteroskedasticity and Autocorrelation Consistent Covariance Matrix", *Econometrica*.
- Carr, P. and Wu, L. (2009), "Variance Risk Premiums", *Review of Financial Studies*. Bollerslev, T., Tauchen, G. and Zhou, H. (2009), "Expected Stock Returns and Variance Risk Premia", *Review of Financial Studies*. Both document the VRP in US markets.

The 4 June 2024 event study, step by step

The event. Votes in the 2024 Lok Sabha election were counted on Tuesday 4 June 2024. Exit polls released on Saturday 1 June predicted a large majority for the ruling alliance, and markets reacted on Monday 3 June. The actual result was a surprise: the ruling party lost its single-party majority, though its alliance still formed the government. This is a textbook high-uncertainty event because everyone knew the date in advance but not the outcome.

Step 1: define "normal". The benchmark is January to April 2024, before the run-up. Median India VIX was 13.8, a 30-day ATM straddle cost 3.2% of spot, and the median daily Nifty move was 0.56%.

Step 2: measure option prices through the event. Two modelled prices are tracked every day from April to mid-July 2024:

- The **30-day ATM straddle as a % of spot**. It always has 30 days left, so it isolates the change in volatility from the passage of time.
- A **real contract**: the NIFTY 27-June-2024 22,500 put (June's monthly expiry, a Thursday then). It shows what a hedger holding one specific option would have seen, including time decay and the effect of Nifty's level.

Date	What happened	Nifty move	India VIX	Straddle (% spot)	22,500 put (points)
April (low)	Calm before the run-up		10.2	about 2.4	
27 May	Run-up		23.2	5.3	409
31 May	Last close before exit polls		24.6	5.6	545
3 Jun	Exit polls predict a big majority	+3.25%	20.9	4.8	217
4 Jun	Result: no single-party majority	−5.93% (−8.5% intraday)	26.8	6.1	903
5 Jun	Relief rally, volatility crush	+3.36%	18.9	4.3	335
7 Jun	Back to near normal		16.9	3.9	112

Step 3: read the pattern. Four phases appear, and they are typical of scheduled events.

1. **Build-up.** VIX more than doubled over six weeks while Nifty stayed between 22,000 and 23,000. The market paid for the event, not for any trend.
2. **False relief.** The exit-poll rally knocked VIX down 15% and the put lost 60% of its value the day before it was needed most.
3. **Shock.** The put was worth 4.2 times its previous close on result day.

4. **Crush.** VIX fell 29% the next day, and within three days option prices were back near normal. Anyone who did not sell on 4 June lost most of the gain.

Step 4: back out the move the market expected. On a normal day, VIX ÷ √252 gives the expected one-day move: 24.6 ÷ 15.9 ≈ 1.55%. But that spreads the whole month's variance evenly, while the market knew the risk was concentrated on one day. The fix uses the fact that variances add up. VIX covers 30 calendar days. Suppose 29 of them are normal days at the base level (median VIX in Q1 2024 = 14.4) and one is the event day. Then:

```
VIX_pre² × 30 / 365  =  VIX_base² × 29 / 365  +  σ_event²

⇒  σ_event = √[ (VIX_pre² × 30 − VIX_base² × 29) / 365 ]
```

With the 31 May VIX of 24.6: $\sqrt{((24.6^2 \times 30 - 14.4^2 \times 29) \div 365)} = 5.8\%$. With the 3 June VIX of 20.9 it gives 4.4%. The actual move was 5.9%. So the options market priced the size of the event well. It just did not know the direction, and the exit polls briefly made it complacent.

Step 5: compare with other high-uncertainty days. The same measures for five other shocks, plus a normal day. "Move vs implied" is the absolute Nifty move divided by the previous day's VIX ÷ √252.

Day	Nifty move	VIX before → after	Straddle before → after (% spot)	Move vs implied
COVID crash, 23 Mar 2020	−12.98%	67.1 → 72.0	15.3 → 16.4	3.3×
Ukraine invasion, 24 Feb 2022	−4.78%	24.5 → 32.0	5.6 → 7.3	3.2×
Exit polls, 3 Jun 2024	+3.25%	24.6 → 20.9	5.6 → 4.8	2.1×
Election result, 4 Jun 2024	−5.93%	20.9 → 26.8	4.8 → 6.1	4.6×
Yen-carry unwind, 5 Aug 2024	−2.68%	14.3 → 20.4	3.3 → 4.7	3.0×
US tariff crash, 7 Apr 2025	−3.24%	13.8 → 22.8	3.2 → 5.2	3.8×
Normal day (median)	0.47% (absolute)	14.7, about ±2%	3.4	0.5×

Two differences stand out. The election was the only shock that arrived with VIX already high, because it was scheduled. The unscheduled shocks of 2024 and 2025 hit when VIX was near 14, which is when protection was cheap. And the election produced the largest move relative to the naive implied move (4.6×), because the naive measure spreads the event risk across the month.

Step 6: generalise. The script also takes the 20 largest one-day VIX jumps in the sample. On those days VIX rose 28% on average, Nifty fell 3.2% and the 30-day straddle rose by 1.3% of spot. On all other days the average absolute Nifty move was 0.7%.

What this means for the hedge. Protection should be bought before the build-up, not during it, and sold into the build-up or the shock, not after the crush. The proposed strategy did exactly that in 2024. It bought a 21,350 put on 1 April 2024 for 195 points, sold half on 27 May at VIX 23 for 177 and the rest on 5 June for 95. It lost a little, because Nifty closed above 21,350 even on result day. Even well-timed insurance only pays when the fall is deeper than the deductible.

How the hedge was designed, including the versions that were rejected

The assignment fixed the idea: buy protection in low-VIX regimes and monetise it in spikes. Everything else was a design choice. Each choice below says what was picked, why, and what the alternatives did in the backtest.

The final rules.

Rule	Setting
Instrument	Nifty put, strike 5% below spot (rounded to 50), on 100% of the portfolio
Tenor	3 months
Entry	First trading day of each month, only if India VIX closes below 13
Roll	If VIX is still below 13 next month, sell the old put and buy a fresh 3-month one
No new buying	If VIX is 13 or above on the roll date, keep the existing put; buy nothing
Monetise half	Sell 50% the first day VIX closes at 22 or above
Monetise the rest	Sell the remaining 50% once VIX closes 20% below its highest close since crossing 22

Rule	Setting
Proceeds	Reinvest in Nifty the same day

Choice 1: a put, not a put spread or VIX futures. A put spread (buy the 95% put, sell an 85% put) is cheaper, but it stops paying once Nifty falls below 85%. The always-on spread had a −30% maximum drawdown against −20% for the plain put, because COVID fell straight through the short strike. India VIX futures exist but trade too thinly to use.

Choice 2: strike 5% out of the money. The strike sets the deductible. An ATM put actually did better here (15.3% a year, −18.6% drawdown) and a 10% OTM put did worse (13.0%, −24.9%). 5% OTM was kept as the middle ground between cost and protection, and because it is the conventional retail and institutional choice. You could defend ATM if you argue the cost difference is small when only buying at low VIX.

Choice 3: buy only when VIX is below 13. 13 is roughly the 30th percentile of VIX in 2020 to 2026. Option prices are lowest in absolute terms there, and that is also where the regime analysis found the least realised volatility. The threshold is not knife-edge. Buying below 12, 14 or 15 all gave 12.2% to 15.7% a year. Buying every month regardless of VIX gave 5.9% to 8.5%.

Choice 4: a 3-month tenor, rolled monthly. This replaced the first design. The first version used 2-month puts. Its January 2020 put expired on 2 March 2020, a week before the COVID crash accelerated. VIX was above 13 at every roll date after that, so the strategy owned nothing during the crash. Its drawdown was −36%, almost the same as unhedged. That was not a flaw of the idea, just bad luck with the calendar, but it showed the hedge must survive a gap between a calm signal and a crash. A 3-month put bought on 2 January 2020 lives until 1 April 2020 and covers the whole crash. Longer tenor also decays more slowly per month (see the Greeks in Theory 3). 1-month and 2-month versions both returned about 10.4% with −36% to −39% drawdowns. 3-month and 6-month versions returned about 13.7% with about −22%.

Choice 5: the monetisation rule. Five versions were compared, all with 3-month puts.

Rule	How it sells	CAGR	Max drawdown	COVID window	CAGR from July 2020
Hold to expiry	Never sells early	17.2%	−17.2%	−6.3%	15.3%
Trailing	Sells everything once VIX is 20% below its post-22 peak	16.5%	−17.2%	−6.3%	15.3%
Hybrid (proposed)	**Half at VIX 22, the rest on the 20% fade**	**13.9%**	**−22.0%**	**−21.1%**	**14.9%**
Ladder 22 / 30	Half at VIX 22, the rest at VIX 30	11.9%	−33.9%	−33.9%	14.8%
Sell all at 22	Everything at VIX 22	11.0%	−35.9%	−35.9%	14.4%

Why the hybrid, when two rules beat it? Be ready for this question. In this sample, holding was best because the only true crash kept going after VIX first crossed 22. On 28 February 2020 VIX hit 23 with Nifty at 11,202, and Nifty still had another 32% to fall (to 7,610 on 23 March). Every rule that sold everything early lost most of the protection. But the opposite happens in a short spike. In 2024 the hedge put bought on 1 April for 195 points was worth 177 on 27 May, during the election build-up. Held to expiry on 1 July, it was worth nothing. The hybrid got back about 70% of its cost. Nobody knows on the day VIX crosses 22 which kind of spike it is. The hybrid locks in half the gain in case the spike reverses, and lets half ride in case it becomes a crash. It is chosen to be reasonable in both worlds, not to be best in the one world this sample happened to contain. Over July 2020 to April 2026, without COVID, all five rules land within 1% a year of each other.

Choice 6: one 3-month put, not overlapping tranches. Staggering means buying a third of the hedge each month, so three puts with different expiries are always alive. It smooths the calendar risk from Choice 4. In the backtest it did worse: 11.8% and −32.7% for the hybrid version, because in January 2020 only a third of the hedge had been bought when VIX stopped being low. 6-month sixths were worse again. Worth revisiting with a longer sample.

Choice 7: reinvest the proceeds. Insurance pays out when Nifty is cheap. Putting the money straight back into Nifty means the hedge buys the dip, which the regime analysis supports (high-VIX days were followed by +3.4% average 21-day returns).

How to run it for real. On the first trading day of each month, check India VIX. If it is below 13, buy Nifty puts 5% below spot expiring in about 3 months. Number of lots = portfolio value ÷ (Nifty level × lot size). Check NSE's current Nifty lot size, which has changed several times. Liquidity is good in the near months, thinner three months out, so expect a wider spread than the 0.5 vol points assumed. Then watch the daily VIX close for the 22 trigger and the 20% fade.

How the backtest works, and what each metric means

A backtest replays history day by day and applies the strategy's rules as if you had been trading. This one runs over 1,536 trading days from 2 January 2020 to 13 April 2026 (6.28 years).

The portfolio. It starts with wealth of 1.0, all in Nifty. The code holds it as a number of index units: units = 1 ÷ Nifty close on day one. There is no cash account. When the strategy buys a put, it sells enough Nifty units to pay the premium. When it sells a put, or a put expires in the money, the money buys Nifty units the same day. So the portfolio is always fully invested, and the hedge cost shows up directly as fewer units.

The option positions. Each open option is a small record: strike K, expiry date, quantity, and sign (+1 for bought, −1 for sold, used only by the put spread). Quantity is measured in index units and set equal to the whole portfolio value ÷ Nifty, so the put covers 100% of the portfolio.

Dates. Roll dates are the first trading day of each month. An option opened on a roll date expires on the roll date 1, 2, 3 or 6 months later. These are synthetic expiries, not NSE's real last-Thursday (now Tuesday) expiries. That keeps the code simple, and the limitations note says how much expiry timing mattered.

Pricing inside the backtest. Every option is valued with Black-Scholes. Its volatility is VIX + 0.6 × (percent out of the money). Buying uses that plus 0.5 vol points and selling uses it minus 0.5, so every round trip pays a spread. Positions are marked each day at the middle (no spread). On its expiry date an option is worth its intrinsic value max(K − S, 0).

What happens each day, in order.

1. **Settle expiries.** Any option whose expiry date has arrived is closed at intrinsic value, and the cash goes into Nifty.
2. **Monetisation checks** (only for the timed strategies that monetise). Compare today's VIX close with the triggers and sell all or part of each put as the rule says.
3. **Roll or open.** On a roll date, the always-on strategies buy their new option. The timed strategies do so only if VIX is below 13; if so they first sell the old put.
4. **Record.** Value = Nifty units × Nifty close + market value of the options. Also record how much notional is hedged.

All trades happen at the day's closing prices, using the same day's VIX close as the signal. In real life you would trade just before the close or the next morning, so the backtest is slightly optimistic about execution.

The seven strategies.

Name	What it does
Unhedged	Holds Nifty, nothing else. The benchmark.
Always 1M 95% put	Every month buys a 1-month 5% OTM put and holds it to expiry. Classic always-on insurance.
Always 1M 95/85 put spread	Same, but also sells a 15% OTM put to cut the cost.
Always 3M 95% put, rolled monthly	The proposed instrument bought every month, whatever VIX is. Isolates the value of the VIX timing.
VIX-timed 3M put, hold	Buys only when VIX is below 13, never sells early.
VIX-timed 3M put + hybrid monetisation	**The proposal.**
VIX-timed 3M put, sell all at VIX 22	The simplest monetisation rule.

Performance metrics. Here V is the portfolio value series and r its daily returns.

Metric	Formula	How to read it
Final	V at the end, starting from 1.0	1.94 means ₹1 grew to ₹1.94
CAGR	(V_end ÷ V_start)^(1 ÷ years) − 1	Average yearly growth, compounded
Volatility	sd(r) × √252	How bumpy the ride was
Sharpe	(CAGR − 6.5%) ÷ volatility	Return above the risk-free rate per unit of risk. The code uses CAGR in the numerator, a slightly conservative variant of the textbook version, which uses the mean return.
Sortino	(CAGR − 6.5%) ÷ downside deviation	Like Sharpe, but only penalises falls. Downside deviation is sd of the negative daily returns × √252.
Max drawdown	Lowest value over time of V ÷ (running peak of V) − 1	The worst fall from a high point. The number a nervous investor feels.

Metric	Formula	How to read it
Calmar	CAGR ÷ size of the max drawdown (ignoring its minus sign)	Return per unit of worst-case pain
Worst day, worst 21 days	Lowest 1-day and 21-day return	Tail risk over short windows

Hedge cost metrics. Every option cash flow is written to a ledger (the `ledger_*.csv` files) with date, action, strike, quantity, price, cash and VIX. Each cash flow is converted to a percentage of the portfolio value that day.

- **Premium paid p.a.** Sum of all money spent buying options, as % of the portfolio, divided by 6.28 years.
- **Recovered p.a.** Sum of all money received from sales and expiries, on the same basis.
- **Net hedge cost p.a.** Paid minus recovered. Negative means the options made money overall.
- **Time hedged.** Share of days with at least one long put open.

Cost and payoff by VIX regime, two ways.

- **By regime at purchase.** Each option's whole life (premium out, all money back in) is attributed to the regime on the day it was bought. This answers "is protection bought in low VIX a good deal?"
- **By regime of the day.** Each day's strategy return minus the unhedged return is summed by that day's regime. The regime is taken from the previous close, so the classification uses only information available at the start of the day. This answers "in which regimes does the hedge cost money, and in which does it pay?" The script reports the cumulative total and an annualised rate (total ÷ days in that regime × 252).

Stress episodes. Returns of every strategy over six windows: the COVID crash, the 2022 war and rate hikes, election result day, the August 2024 yen-carry unwind, the September 2024 to March 2025 correction and the April 2025 tariff shock.

Robustness runs. The same backtest re-run many times with one setting changed: buy threshold × first monetisation trigger (24 runs), skew × spread (18 runs), tenor (1, 2, 3, 6 months), strike (90%, 95%, 100%), and monetisation rule × tenor structure (20 runs, each also measured from July 2020 to exclude COVID).

Results, and how to interpret them honestly

Each result below comes with what it supports and what it does not. Examiners reward the second part as much as the first.

1. The volatility risk premium

India VIX averaged 17.3 against 15.1 realised over the following 21 days: a premium of 2.2 vol points (Newey-West t = 2.8), positive on 78% of days. By regime it was 2.1 points in Low, 1.4 in Normal and 4.0 in High.

- **Supports:** buying options is, on average, a losing trade, and the loss is worst right after a shock. That is the economic reason to buy protection only when it is cheap, and to sell it into fear.
- **Does not support:** "always sell options". The 22% of days with a negative VRP include the crashes, when realised volatility exceeded VIX by as much as 64 points (5 March 2020; the VRP chart cuts off at −35). Selling options collects small premiums and occasionally loses a lot.

2. VIX regimes and Nifty returns

Regime	Days	Same-day vol (ann.)	Worst day	Next 21 days: mean (NW t)	Next 63 days: mean	Next 63 days: 5th pct
Low (below 13)	408	9.0%	−1.4%	+1.1% (1.8)	+3.9%	−9.0%
Normal (13 to 20)	737	13.2%	−2.2%	−0.2% (−0.2)	+0.9%	−14.6%
High (above 20)	391	30.1%	−13.9%	+3.4% (3.8)	+9.5%	−7.4%

- **Supports:** VIX regimes sort risk very clearly (Levene $p < 0.001$). High-VIX periods were followed by strong rebounds in this sample. The worst forward outcomes started in the Normal regime, as VIX rose from low levels, which is why protection has to be in place before the regime changes.
- **Does not support:** a market-timing rule. Same-day returns did not differ by regime (Kruskal-Wallis $p = 0.66$). The high-regime rebound is mostly one event, the 2020 recovery.

3. The 2024 election

Premiums doubled over six weeks, the exit polls cut them by 15% a day before the shock, and VIX fell 29% the day after. The event-variance method implied a 4.4% to 5.8% result-day move against the actual 5.9%.

- **Supports:** the options market prices scheduled events in advance and prices their size well.

- **Does not support:** a claim about real traded option prices. The straddle and the 22,500 put are modelled from VIX. Real prices around the event would have had steeper skew and term-structure effects.

4. The backtest

Strategy	CAGR	Vol	Sharpe	Max DD	Premium paid p.a.	Net hedge cost p.a.	Time hedged	CAGR from Jul 2020
Unhedged	11.1%	18.3%	0.25	−38.4%	0	0	0%	15.4%
Always 1M 95% put	8.3%	13.2%	0.14	−19.6%	7.5%	3.0%	99%	9.6%
Always 1M 95/85 spread	7.3%	15.0%	0.05	−30.2%	9.5%	3.6%	99%	10.1%
Always 3M 95% put	6.3%	11.7%	−0.02	−17.9%	20.2%	4.9%	97%	7.6%
VIX-timed 3M, hold	17.2%	14.6%	0.74	−17.2%	2.9%	−4.4%	47%	15.2%
VIX-timed 3M, hybrid (proposed)	**13.9%**	**15.4%**	**0.48**	**−22.0%**	**2.9%**	**−2.2%**	**45%**	**14.9%**
VIX-timed 3M, sell all at 22	11.0%	17.7%	0.25	−35.9%	2.9%	0.2%	43%	14.4%

The "always 3M" line pays 20% a year in premium because it buys a fresh 3-month put every month and sells the old one, so gross flows are large. Its net cost of 4.9% is the number that matters.

Where the money came from. The proposed strategy made 22 trades. Only 5 made money and 3 were monetised early. Two did most of the work.

- **January 2020.** Bought at VIX 11.5 for 93 points (0.8% of the index). Half sold on 28 February at VIX 23 for 546 points. The rest sold on 31 March at VIX 64 for 3,050 points, 33 times the cost.
- **January 2026.** Bought on 2 January at VIX 9.5 (strike 25,000) for 134 points. Nifty then fell 15% to 22,331 by 30 March. Half was sold on 9 March at VIX 23 for 1,113 points, and the other half expired on 1 April worth 2,321 points. The hedge cut that sell-off's drawdown from 15% to 10%.

By regime. Options the proposal bought (always in Low VIX) returned a net +2.2% of the portfolio a year over their lives. Always-on monthly puts lost 0.7%, 1.7% and 0.6% a year on those bought in Low, Normal and High VIX. Day by day, the proposal cost 2.1% a year relative to unhedged on low-VIX days (the insurance premium), and earned 2.0% a year on normal days and 6.3% a year on high-VIX days.

5. The honest bottom line

- **Robust:** buying only in low VIX beats buying always. That held at every threshold, skew, spread, tenor and rule tested, and from July 2020 as well as from January 2020.
- **Robust:** always-on protection is expensive. It cost 3% to 5% a year net and lowered CAGR by 3 to 5 points.
- **Fragile:** the size of the gain. Almost all of the proposal's advantage over unhedged comes from one cheap put that happened to be alive in March 2020. From July 2020 the proposal returned 14.9% against 15.4% unhedged, about 0.5% a year of insurance cost. It softened the early-2026 fall, but its maximum drawdown after 2020 (−17.2%, the slow 2024–25 correction) was the same as unhedged.
- **Fragile:** the choice of tenor and monetisation rule. 2-month puts or selling everything at VIX 22 would have missed COVID.
- **Therefore:** present it as cheap crash insurance, roughly 0.5% a year in calm years, that pays off hugely in the rare crash that starts from calm. Do not present it as a strategy that beats the market.

Ideas to build on top of this project

The project's two biggest weaknesses are modelled option prices and a short sample with one crash. The most valuable extensions fix those. They are ordered from most to least impact.

1. Use real option prices. Download NSE's daily F&O bhavcopy files for the dates you need (on your own machine; see the data section). Each file lists every Nifty option's strike, expiry, close, settlement price and open interest. Then:

- compute the real ATM implied volatility each day by inverting Black-Scholes (`scipy.optimize.brentq` on `bs_put` minus the market price);
- measure the real skew (IV of the 95% put minus ATM IV) and the real term structure (3-month IV minus 1-month IV), and see how both change across VIX regimes;
- re-run the backtest buying and selling at real settlement prices. This is the single most convincing upgrade, because it removes the main limitation.

2. Test on a longer, independent sample. India VIX history goes back to 2008 on NSE. Freeze the rules exactly as they are (13, 22, 20% fade, 3 months, 5% OTM) and run them over 2008 to 2019. That period has the 2008 crisis, the 2011 eurozone crisis, the 2013 taper tantrum, the 2015–16 sell-off and 2018. If the rules still help there, the result is out of sample and far more credible.

3. Include dividends. Swap the Nifty price index for the Nifty 50 Total Returns Index. Every strategy's CAGR rises by roughly the dividend yield. The comparison should barely change, but it removes a question.

4. Add realistic costs. Model next-day execution instead of same-day close, NSE's actual expiry calendar (last Thursday until 2025, now Tuesday), securities transaction tax on option sales, brokerage, exchange fees and GST. Widen the half-spread for 3-month options, which trade less than near-month ones.

5. Smarter entry signals.

- **Adaptive thresholds.** "Buy when VIX is below its own 30th percentile over the past year" instead of a fixed 13. VIX levels drift across decades, so a fixed number may not travel.
- **VRP-based entry.** Buy when VIX is low relative to recent realised volatility (options cheap relative to actual risk), not just low in level.
- **Forecast models.** Fit a GARCH or HAR-RV model to Nifty returns to forecast next-month volatility, and compare the forecast with VIX. A large gap is a signal about whether options are rich or cheap.
- **Regime-switching models.** A hidden Markov model on returns lets the data choose the regimes instead of you choosing 13 and 20.

6. Other hedge structures. A collar (buy a put, sell an OTM call to pay for it) costs almost nothing but caps upside. A put ratio or a far-OTM tail hedge (10% to 20% below spot) costs less but pays only in big crashes. Hedging 50% of the portfolio instead of 100% shows the cost-protection trade-off directly.

7. Many outcomes instead of one path. A block bootstrap resamples chunks of 2020–26 history to create thousands of alternative histories, and gives a distribution of hedge results instead of one number. It directly answers "was it just COVID?".

8. A scheduled-event study across many events. Repeat the election analysis for every Union Budget, RBI policy decision and US Fed decision since 2020. Measure how much VIX builds up before each and how much it drops after. That tells you whether buying protection before known events is systematically over- or under-priced.

9. Sentiment combinations. Combine India VIX with the Nifty put-call ratio, foreign investor flows or the US VIX. A simple question with a clean answer: does US VIX lead India VIX by a day (Granger causality test)?

10. A live dashboard. A small Streamlit app that loads today's Nifty and VIX, shows the current regime, and says whether the rule would buy, hold or sell. Good for the presentation demo and a natural end product.

Presenting it: likely questions and good answers

A good answer admits the limitation in the first sentence, then says why the conclusion still stands. Each answer below is short enough to say aloud.

"Why didn't you use real option prices?" Historical NSE option data was not accessible for this study, so premiums are Black-Scholes prices using India VIX plus a put-skew add-on and a bid/ask spread. India VIX is itself computed from real Nifty option prices, so the level is market-based; the skew and term structure are assumptions. I stress-tested skew from 0 to 1 vol point per 1% OTM and spreads up to 1 vol point, and the ranking of strategies did not change.

"VIX is a 30-day measure. Is it right to price 3-month options with it?" No, it is an approximation, and it probably flatters the strategy: in calm markets real 3-month IV is above 30-day VIX, so real puts cost more; in a spike 3-month IV rises less than VIX, so they sell for less. It is the main limitation, and real option data is the first thing I would add.

"Why 13 and 20? Isn't that data mining?" They are roughly the 30th and 80th percentiles of VIX in the sample, so yes, they were set on the same data. That is why the guide tests buy thresholds from 12 to 16: every one of them beat buying regardless of VIX by roughly 4 to 10 points a year. The conclusion does not depend on 13 exactly.

"Insurance should cost money. How did your hedge make money?" Over the whole period, one put bought in January 2020 for 0.8% of the index was sold during the COVID crash for 33 times its cost. Without that crash, from July 2020, the hedge cost about 0.5% a year, which is what insurance should cost. I present it as cheap insurance, not as a return source.

"So the result is just COVID?" The size of the gain is mostly COVID, yes. Two things are not: buying only in low VIX beat always-on hedging in every test, including after COVID (14.9% against 7.6% to 10.1%). And the January 2026 trade also paid off, cutting that sell-off's drawdown from 15% to 10%.

"Holding to expiry did better. Why propose the hybrid?" Holding was best here because the one crash kept getting worse after VIX crossed 22. In a short spike, like the 2024 election, a held put gives back everything; the hybrid recovered 70% of its cost there. On the day VIX crosses 22 you cannot know which kind of spike it is, so the hybrid takes half the gain and lets half ride. It is the robust choice, not the in-sample best.

"Why does VIX go up when the market falls?" Investors rush to buy puts when prices fall, which pushes up implied volatility, and falling prices also raise leverage and real risk. In the data a 1% Nifty fall came with a 2.7% rise in VIX, and the correlation was −0.56.

"Does a high VIX mean the market will fall?" No. VIX predicts the size of moves, not the direction. After high-VIX days Nifty actually rose 3.4% on average over the next month, because fear was overpriced and the market recovered.

"Why is implied volatility usually above realised?" Option sellers are selling crash insurance and demand a premium for the risk of rare large losses, and hedgers are willing to pay it. That gap is the volatility risk premium: 2.2 points on average here, and 4 points after shocks.

"What is Newey-West and why did you need it?" Forward 21-day returns on consecutive days overlap by 20 days, so they are not independent observations. Ordinary standard errors would overstate significance. Newey-West standard errors correct for that overlap; I set the lag equal to the horizon.

"Your VIX replication gave 16.1, not 15. Is the code wrong?" No, that is the point of the test. With every option at 15% it returns exactly 15.0, which confirms the formula. With 15% at the money plus put skew it returns 16.1, because the formula weights low-strike puts by $1/K^2$. So VIX includes the price of crash protection, not just expected volatility.

"Could you have profited from the election?" Only by owning protection before the build-up. The modelled June 22,500 put cost about 284 points on 25 April 2024 at VIX 10.7. It was worth 545 by 31 May and 903 on result day. Bought on 31 May instead, it lost 60% after the exit polls and only paid on result day itself; VIX fell 29% the next morning.

"Why did the put spread do worse than the plain put?" It stops paying below the short strike, 15% under spot. COVID fell about 38%, straight through it, so the spread protected far less exactly when it mattered.

"How would a fund actually implement this?" On the first trading day of each month, check VIX. If it is below 13, buy listed Nifty puts 5% below spot, about 3 months out, on the portfolio's notional. Watch the daily VIX close: sell half at 22, the rest when VIX is 20% off its peak. It is mechanical, so it can run off a daily data feed.

"Your Sharpe ratio uses CAGR. Is that standard?" The textbook Sharpe uses the average return. Using CAGR is slightly more conservative and is applied the same way to every strategy, so comparisons are fair.

"What would you do with more time?" Three things, in order: real option prices from NSE bhavcopy files, an out-of-sample test on 2008 to 2019 with the rules frozen, and realistic costs with next-day execution.

Glossary

Term	Meaning
Annualise	Convert a daily figure to a yearly one. For volatility multiply by $\sqrt{252}$; for returns compound over 252 days.

Term	Meaning
ATM, OTM, ITM	At, out of, in the money. A put is OTM when its strike is below spot, ITM when above.
Backtest	Replaying history day by day and applying a strategy's rules as if trading.
Bhavcopy	NSE's daily file with the closing and settlement prices of every listed contract.
Black-Scholes(-Merton)	Formula for the fair price of a European option from spot, strike, time, rates, dividends and volatility.
CAGR	Compound annual growth rate: the constant yearly return that turns the start value into the end value.
Calmar ratio	CAGR divided by the size of the maximum drawdown.
Collar	Buying a put and selling a call to pay for it. Cheap protection with capped upside.
Convexity	Gains that accelerate as the move gets bigger. Long options have it.
Data snooping	Choosing parameters by looking at the same data used to test them, which flatters results.
Delta, gamma, vega, theta	Option sensitivities to spot, to changes in delta, to implied volatility and to the passage of time.
Drawdown	Fall from the highest previous value. Maximum drawdown is the worst such fall.
Event variance	The extra variance a known event adds, backed out of the rise in VIX before it.
Expiry	The date an option stops trading and settles.
Fear index	Nickname for VIX, because it rises when investors rush to buy protection.
Forward (F)	The fair future price of Nifty for an expiry, S × e^((r − q)T).
Hit rate	Share of observations with a positive return.
Implied volatility (IV)	The volatility that makes Black-Scholes match an option's market price.

Term	Meaning
India VIX	NSE's index of expected 30-day Nifty volatility, computed from Nifty option prices.
Intrinsic value	What an option is worth if exercised now. For a put, max(K − S, 0).
Kruskal-Wallis test	Rank-based test of whether several groups come from the same distribution.
Leverage effect	Volatility rises more when prices fall than it falls when prices rise.
Levene's test	Test of whether several groups have equal variance.
Log return	ln(P_t ÷ P_{t−1}). Additive over time, close to percentage return for small moves.
Look-ahead bias	Using information in a backtest that would not have been known at the time.
Mean reversion	Tendency to drift back towards a long-run average. VIX shows it strongly.
Mincer-Zarnowitz regression	Outcome regressed on forecast. An unbiased forecast gives intercept 0 and slope 1.
Model-free variance	Variance computed directly from a strip of option prices, without assuming a pricing model. The basis of VIX.
Monetise	Sell a hedge that has gained value, turning the paper gain into cash.
Newey-West (HAC)	Standard errors that stay valid when observations overlap or are autocorrelated.
Notional	The amount of exposure an option covers, here the whole portfolio value.
Premium	The price paid for an option.
Protective put	Owning an asset plus a put on it, which puts a floor under losses.
Put-call parity	C − P = S·e^(−qT) − K·e^(−rT) for European options with the same strike and expiry.

Term	Meaning
Put spread	Buying one put and selling a lower-strike put. Cheaper, but protection stops at the lower strike.
Realised volatility (RV)	Standard deviation of actual returns over a window, annualised.
Regime	A period grouped by VIX level: Low below 13, Normal 13 to 20, High above 20.
Roll	Sell an existing option and buy a new one with a later expiry.
Sharpe ratio	Return above the risk-free rate divided by volatility.
Skew (smile, smirk)	Implied volatility differing by strike. In equity indices, OTM puts carry the highest IV.
Sortino ratio	Like Sharpe, but divides by downside deviation only.
Straddle	An ATM call plus an ATM put. Its price measures the expected size of a move.
Strike (K)	The price at which an option can be exercised.
Tenor	Time from purchase to expiry, here 3 months.
Term structure	Implied volatility by expiry. Normally upward-sloping, inverted in crises.
Time value	Premium minus intrinsic value. It decays to zero by expiry.
Vol crush	The sharp fall in implied volatility once an event's outcome is known.
Vol point	One percentage point of annualised volatility, for example VIX from 12 to 13.
VRP	Volatility risk premium: implied minus subsequently realised volatility.

Code walkthrough: fdrm_vix_analysis.py

0. How to run it, and the map of files

The whole analysis is one Python script of 726 lines, `fdrm_vix_analysis.py`. It reads two CSV files, prints its key numbers as it goes, writes every table to `output/`, every chart to `figures/` and every quoted number to `output/results.json`. The deck and the report were written from those files. This tab goes through the script block by block, quoting the code and explaining each line.

Run it.

```
pip install pandas numpy scipy statsmodels matplotlib
cd fdrm-vix
python fdrm_vix_analysis.py
```

It runs in about 30 seconds, most of it the 70-odd backtests in the robustness section. Re-running it from scratch reproduces `results.json` exactly; that was checked while writing this guide. It was built with Python 3.11, pandas 3.0, NumPy 2.4, SciPy 1.17, statsmodels 0.15 and matplotlib 3.11. Older versions should work, but if a number differs, check versions first.

The notebook. `fdrm_vix_analysis.ipynb` is the same code split into cells, already run, with every printout and chart visible. Open it in Jupyter or VS Code to read the outputs without running anything. The script marks cell boundaries with `# %%` comments and text cells with `# %% [markdown]`. That is the "percent" format used by the jupytext tool, which turned the script into the notebook. If you change the code, change the `.py`, re-run it, and regenerate the notebook with `jupytext --to ipynb fdrm_vix_analysis.py` followed by `jupyter nbconvert --to notebook --execute --inplace fdrm_vix_analysis.ipynb`.

What the script produces, by section.

Section	Output files
1. Load and align	`output/nifty_vix_merged.csv` (the cleaned, joined daily data with every derived column)
2. VIX formula check	Numbers only (in `results.json`)
3. VRP	`output/vrp_by_regime.csv`, `figures/fig2_vrp.png`

Section	Output files
4. Regimes	`output/returns_by_regime.csv`, `output/sameday_by_regime.csv`, `figures/fig1_nifty_vix_regimes.png`, `figures/fig3_regime_returns.png`
5. Election	`output/election_window.csv`, `output/event_comparison.csv`, `figures/fig4_election_window.png`, `figures/fig5_put22500.png`
6. Backtest	`output/backtest_values.csv` (daily value of all 7 strategies), `output/backtest_performance.csv`, `output/ledger_s1..s6_*.csv` (every option trade), `output/hedge_cost_payoff_by_regime.csv`, `output/excess_return_by_regime_*.csv`, `output/episode_returns.csv`, `figures/fig6_backtest_equity.png`, `figures/fig7_cost_by_regime.png`
7. Robustness	`output/sensitivity_thresholds.csv`, `output/sensitivity_pricing.csv`, `output/sensitivity_rule_by_tenor.csv`, `figures/fig8_sensitivity.png`
End	`output/results.json`

A quirk worth knowing: the ledgers are numbered by the strategy's position in the list, so `ledger_s3_timed_hold.csv` is "Always 3M put" (which reuses the timed-hold code with the VIX filter switched off) and `ledger_s4_timed_hold.csv` is the real "VIX-timed, hold". The unhedged strategy has no trades, so there is no s0 ledger.

How to change something. Every assumption is a constant near the top (section 1 of this tab). Change it, re-run, and every table, chart and `results.json` updates. The report and the deck do not update themselves; their numbers would need re-copying.

1. Imports, folders and assumptions (lines 1 to 72)

The file opens with a markdown cell (lines 1 to 18) describing the data and the modelling choice. Then the imports:

```
import json                      # to write results.json
import math                      # sqrt, exp on single numbers
from pathlib import Path         # file paths that work on any OS

import matplotlib
matplotlib.use("Agg")            # draw charts to files, no screen needed
import matplotlib.dates as mdates
import matplotlib.pyplot as plt
import numpy as np               # fast maths on whole arrays
import pandas as pd              # tables indexed by date
import statsmodels.api as sm     # regressions with Newey-West errors
from scipy import stats          # Kruskal-Wallis and Levene tests
from scipy.stats import norm     # normal distribution N(.) for Black-Scholes
```

`matplotlib.use("Agg")` must come before `pyplot` is imported. It picks a backend that renders straight to image files, so the script also runs on a server with no display.

```
ROOT = Path(__file__).resolve().parent if "__file__" in globals() else
Path.cwd()
DATA, FIG, OUT = ROOT / "data", ROOT / "figures", ROOT / "output"
FIG.mkdir(exist_ok=True)
OUT.mkdir(exist_ok=True)
```

- `ROOT` is the folder the script lives in. When the code runs as a script, `__file__` is its path. Inside a notebook `__file__` does not exist, so it falls back to the current working directory. This one line makes the same code work both ways.
- `/` on `Path` objects joins folder names, so `ROOT / "data"` is the data folder.
- `mkdir(exist_ok=True)` creates the output folders if missing and does nothing if they exist.

The assumptions block. Every modelling choice is a named constant, so it can be changed in one place and re-tested.

```
R = 0.065            # risk-free rate
Q = 0.013            # Nifty dividend yield
SKEW_PER_PCT = 0.6   # extra put IV per 1% OTM
HALF_SPREAD = 0.5    # bid/ask half-spread in vol points
LOW_VIX, HIGH_VIX = 13.0, 20.0
SPIKE_1, SPIKE_2 = 22.0, 30.0
TRAIL = 0.20
TENOR = 3
PUT_MONEYNESS = 0.95
SPREAD_SHORT = 0.85
TRADING_DAYS = 252
```

Constant	Value	Why this value
`R`	6.5%	About the 91-day Treasury bill yield over the period. Enters Black-Scholes and the Sharpe ratio.
`Q`	1.3%	Roughly Nifty's dividend yield. Lowers the forward price, which makes puts slightly dearer.
`SKEW_PER_PCT`	0.6	A 5% OTM put gets 3 extra vol points, in line with typical Nifty skew. Stress-tested from 0 to 1.0.
`HALF_SPREAD`	0.5	Buy at IV + 0.5, sell at IV − 0.5. Stress-tested from 0 to 1.0.
`LOW_VIX`, `HIGH_VIX`	13, 20	Regime cut-offs, near the 30th and 80th percentiles of VIX in the sample. `LOW_VIX` is also the buy threshold.
`SPIKE_1`	22	First monetisation trigger. VIX closed at or above 22 on about the top 15% of days.
`SPIKE_2`	30	Second rung, used only by the ladder rule.
`TRAIL`	0.20	Sell the rest when VIX is 20% below its post-spike peak.
`TENOR`	3	Months to expiry of the proposed put.
`PUT_MONEYNESS`	0.95	Strike = 95% of spot, so 5% OTM.
`SPREAD_SHORT`	0.85	Short leg of the put-spread baseline, 15% OTM.
`TRADING_DAYS`	252	Trading days per year, for annualising.

Chart styling (lines 53 to 64). `C` is a dictionary of named hex colours so every chart uses the same palette. `REGIME_COL` gives the three regime shading colours (blue for Low, grey for Normal, orange for High). `plt.rcParams.update({...})` sets defaults for all charts: resolution, font size, grey axes, no top and right borders, light grid, no legend frame. None of this affects any number.

```
RESULTS = {}

def save(fig, name):
    fig.tight_layout()
    fig.savefig(FIG / name, bbox_inches="tight", facecolor="white")
    plt.close(fig)
```

- `RESULTS` is an empty dictionary that collects every headline number as the script runs. At the end it is written to `results.json`. This is what keeps the report, the deck and the code consistent.
- `save` tidies the layout so labels don't overlap, writes the PNG with a white background, and closes the figure to free memory.

2. Loading and aligning the data (lines 77 to 102)

```
nifty = pd.read_csv(DATA / "nifty50_daily_raw.csv", parse_dates=
["Date"]).set_index("Date")
nifty = nifty[["Open", "High", "Low", "Close"]].astype(float)
nifty = nifty[~nifty.index.duplicated()].sort_index()
```

- `read_csv(..., parse_dates=["Date"])` reads the file and turns the Date column into real dates. `.set_index("Date")` makes the date the row label, so rows can be looked up by date and two tables can be joined on it.
- The second line keeps four columns (dropping Adj Close and Volume) and forces them to floating-point numbers.
- `nifty.index.duplicated()` is True for any repeated date; `~` flips it, so the third line keeps only the first copy of each date. `sort_index()` puts rows in date order. Neither changed anything for this file, but they protect against a messy download.

```
vix_raw = pd.read_csv(DATA / "india_vix_daily_raw.csv", encoding="utf-8-sig",
parse_dates=["Date"])
vix = vix_raw.rename(columns={"Sum of Open": "VIX_Open", "Sum of High":
"VIX_High",
                              "Sum of Low": "VIX_Low", "Sum of Close":
"VIX"})
vix = vix.set_index("Date")[["VIX_Open", "VIX_High", "VIX_Low",
"VIX"]].astype(float).sort_index()
```

- `encoding="utf-8-sig"` strips the invisible byte-order mark at the start of this file. Without it the first column would be called "Date" and `parse_dates=["Date"]` would fail.
- `rename` maps the Power BI-style names to short ones. The VIX close is simply called `VIX` because it is used everywhere.

```
nifty["ret"] = np.log(nifty.Close).diff()
nifty["rv21_trail"] = nifty.ret.rolling(21).std() * math.sqrt(TRADING_DAYS) *
100
```

- `ret` is the daily log return: the log of today's close minus the log of yesterday's. `.diff()` subtracts the previous row. The first row becomes NaN (not a number) because it has no previous day.
- `rv21_trail` is trailing realised volatility: `.rolling(21).std()` is the standard deviation of the last 21 returns up to and including today. Multiplying by √252 annualises it, and by 100 puts it in percent, the same units as VIX.

```
nifty["rv21_fwd"] = nifty.ret[::-1].rolling(21).std()[::-1].shift(-1) *
math.sqrt(TRADING_DAYS) * 100
```

The most important line in this section. It computes the volatility over the **next** 21 trading days, which is what VIX on day t is trying to forecast. pandas has no forward-looking rolling window, so the line uses a trick:

1. `nifty.ret[::-1]` reverses the series so the latest date comes first.
2. `.rolling(21).std()` on the reversed series covers, for each date t, the return on t and the 20 returns after it (in normal time order).
3. `[::-1]` flips it back to normal order.
4. `.shift(-1)` moves each value up by one row, so the value at t now covers returns t+1 to t+21. Today's return is excluded, because VIX at today's close already knows it.
5. `× √252 × 100` annualises and converts to percent.

The last 21 rows have no full future window, so they are NaN and drop out of the VRP analysis.

```
nifty["fwd21"] = nifty.Close.shift(-21) / nifty.Close - 1
nifty["fwd63"] = nifty.Close.shift(-63) / nifty.Close - 1
nifty["fwd1"] = nifty.ret.shift(-1)
```

- `shift(-21)` pulls the close from 21 rows later onto today's row. Dividing by today's close and subtracting 1 gives the simple return over the next 21 trading days. Same for 63 days (about three months).
- `fwd1` is tomorrow's log return.

All of these are computed on the full Nifty history before the join. If they were computed after, the 16 days missing from the VIX file would make some "one-day" returns span two days.

```
df = nifty.join(vix, how="inner").dropna(subset=["Close", "VIX"])
df["dvix"] = np.log(df.VIX).diff()
df["regime"] = pd.cut(df.VIX, [0, LOW_VIX, HIGH_VIX, 1e9], labels=["Low",
"Normal", "High"])
df.to_csv(OUT / "nifty_vix_merged.csv")
```

- `join(..., how="inner")` matches rows by date and keeps only dates present in both files: 1,536 days from 2 Jan 2020 to 13 Apr 2026. Nifty data from before 2020 is dropped here because there is no VIX for it.
- `dvix` is the daily change in log VIX, roughly the percentage change.
- `pd.cut` puts each day's VIX into a bin. The edges [0, 13, 20, 1e9] (1e9 is just "a very large number") create three bins labelled Low, Normal and High. One detail for inside-out knowledge: bins include their right edge, so a VIX of exactly 13.00 is labelled Low and 20.00 Normal. The backtest's buy rule uses a strict `VIX < 13`. With prices to two decimals, this almost never matters.
- The merged table is saved, so you can open it in Excel and see every derived column.

```
RESULTS["sample"] = dict(start=..., end=..., days=len(df),
                         vix_pct=df.VIX.quantile([.1, .3, .5, .8,
.9]).round(2).to_dict(),

regime_share=df.regime.value_counts(normalize=True).round(3).to_dict())
```

This stores the sample dates, VIX percentiles (10th 11.4, 30th 13.4, median 15.3, 80th 21.0, 90th 24.1) and the share of days in each regime (Low 27%, Normal 48%, High 25%). The percentiles are how the 13 and 20 cut-offs were justified.

3. Black-Scholes functions and the India VIX formula check (lines 115 to 158)

```
def bs_put(S, K, T, sigma, r=R, q=Q):
    T = np.maximum(T, 1e-8)
    sigma = np.maximum(sigma, 1e-4)
    d1 = (np.log(S / K) + (r - q + 0.5 * sigma ** 2) * T) / (sigma *
np.sqrt(T))
    d2 = d1 - sigma * np.sqrt(T)
    return K * np.exp(-r * T) * norm.cdf(-d2) - S * np.exp(-q * T) *
norm.cdf(-d1)
```

- This is the Black-Scholes-Merton put formula from Theory 3, written line for line. `norm.cdf` is N(·), the standard normal cumulative probability.
- The two `np.maximum` lines are guards. A time of exactly zero or a volatility of zero would mean dividing by zero, so they are floored at tiny positive numbers.
- Every operation uses NumPy, so the function works on one number or on whole arrays at once. The election code prices a whole column of dates in one call because of this.
- `r=R, q=Q` are default arguments: if you don't pass a rate or dividend yield, it uses the constants from the top.

```
def bs_call(S, K, T, sigma, r=R, q=Q):
    return bs_put(S, K, T, sigma, r, q) + S * np.exp(-q * T) - K * np.exp(-r
* T)
```

The call is the put plus S·e^(−qT) − K·e^(−rT): put-call parity rearranged. Writing it this way guarantees calls and puts are consistent.

```
def put_iv(vix_level, S, K):
    otm_pct = np.maximum(0.0, (1 - K / S) * 100)
    return (vix_level + SKEW_PER_PCT * otm_pct) / 100
```

The skew model. `(1 − K/S) × 100` is how many percent the strike is below spot. `np.maximum(0, …)` makes it zero for strikes at or above spot. The result is VIX + 0.6 per OTM percent, divided by 100 to turn vol points into a decimal (15 becomes 0.15). The election study uses this; the backtest has its own version with the spread added (section 7).

Re-implementing India VIX.

```
def model_free_var(S, T, iv_fn, r=R, q=Q, step=50):
    F = S * math.exp((r - q) * T)
    Ks = np.arange(round(S * 0.6 / step) * step, round(S * 1.4 / step) * step
+ step, step, dtype=float)
    K0 = Ks[Ks <= F].max()
    Qk = np.where(Ks < K0, bs_put(S, Ks, T, iv_fn(Ks)),
                  np.where(Ks > K0, bs_call(S, Ks, T, iv_fn(Ks)),
                           0.5 * (bs_put(S, Ks, T, iv_fn(Ks)) + bs_call(S,
Ks, T, iv_fn(Ks)))))
    dK = np.gradient(Ks)
    return (2 / T) * np.sum(dK / Ks ** 2 * math.exp(r * T) * Qk) - (1 / T) *
(F / K0 - 1) ** 2
```

This computes the variance for one expiry, following NSE's steps.

- `F` is the forward level. NSE derives it from put-call parity on real quotes; on a synthetic chain it is simply S·e^((r − q)T).
- `Ks` is the strike grid: from 60% to 140% of spot, 50 points apart, like real Nifty strikes. `round(S × 0.6 / 50) × 50` rounds to the nearest 50. `np.arange` excludes its end point, hence the `+ step`.
- `K0` is the highest strike at or below the forward.
- `Qk` picks the option for each strike with nested `np.where(condition, if_true, if_false)`: a put below K0, a call above K0, and the average of both at K0. These are the out-of-the-money options the formula uses. `iv_fn(Ks)` supplies a volatility for every strike, which is how the smile is fed in.

- `dK` is the strike spacing ΔK. `np.gradient` takes half the distance between each strike's two neighbours, which is exactly how the CBOE and NSE define ΔK. Here it is 50 everywhere.
- The return line is the formula from Theory 2: (2/T) × Σ ΔK/K² · e^(RT) · Q(K), minus the small correction (1/T)(F/K0 − 1)².

```
def india_vix_formula(S, iv_fn, d_near=16, d_next=44):
    T1, T2 = d_near / 365, d_next / 365
    v1, v2 = model_free_var(S, T1, iv_fn), model_free_var(S, T2, iv_fn)
    N30 = 30 / 365
    w = (T2 - N30) / (T2 - T1)
    return 100 * math.sqrt((T1 * v1 * w + T2 * v2 * (1 - w)) / N30)
```

- Two expiries: a near month with 16 days left and a next month with 44, a typical mid-month situation.
- `w` is the weight on the near expiry. It is the share of the gap between T1 and T2 that lies beyond 30 days: (44 − 30) ÷ (44 − 16) = 0.5.
- The interpolation blends *total* variance (variance × time), not variance itself, then divides by 30 days to get back a 30-day annualised variance. This is the CBOE/NSE interpolation in years instead of minutes.
- `100 × sqrt(…)` turns the variance into an index level.

```
S0 = 24000.0
flat = india_vix_formula(S0, lambda K: np.full_like(K, 0.15))
skew = india_vix_formula(S0, lambda K: 0.15 + 0.006 * np.maximum(0, (1 - K /
S0) * 100)
                         - 0.002 * np.maximum(0, (K / S0 - 1) * 100))
```

- `lambda K: …` is a one-line unnamed function. Each defines a smile: volatility as a function of strike.
- `flat`: every strike at 15%. Under Black-Scholes with constant volatility, the fair variance is exactly σ², so a correct implementation must return 15. It returns **15.0**, which validates the code.
- `skew`: 15% at the money, +0.6 vol points per 1% below spot for puts, and −0.2 per 1% above spot for calls (the usual smirk shape). It returns **16.1**. The extra 1.1 points come from the 1/K² weighting on expensive low-strike puts, which shows that VIX prices crash protection as well as expected volatility.

4. Volatility risk premium (lines 165 to 196)

```
v = df.dropna(subset=["rv21_fwd"]).copy()
v["vrp"] = v.VIX - v.rv21_fwd
```

- Keep only days that have a full 21-day future window (this drops the last 21 days). `.copy()` makes `v` an independent table so adding columns to it never touches `df`.
- `vrp` is the volatility risk premium for each day: VIX minus the volatility that actually followed. Both are in annualised percent, so the difference is in vol points.

```
vrp_stats = dict(
    mean_vix=v.VIX.mean(), mean_rv_fwd=v.rv21_fwd.mean(),
mean_vrp=v.vrp.mean(), median_vrp=v.vrp.median(),
    share_positive=(v.vrp > 0).mean(),
    corr_vix_rvfwd=v[["VIX", "rv21_fwd"]].corr().iloc[0, 1],
    corr_vix_rvtrail=v[["VIX", "rv21_trail"]].corr().iloc[0, 1],
)
```

- Means and median of VIX, forward RV and VRP (17.3, 15.1, 2.2 and 3.0).
- `(v.vrp > 0)` is a column of True/False. The mean of True/False is the share of True, here 0.78.
- `.corr()` returns a 2×2 correlation matrix; `.iloc[0, 1]` picks the off-diagonal number. VIX correlates 0.61 with future RV and 0.86 with past RV.

```
nw = sm.OLS(v.vrp.values, np.ones(len(v))).fit(cov_type="HAC", cov_kwds=
{"maxlags": 21})
vrp_stats["nw_t_mean_vrp"] = float(nw.tvalues[0])
```

The Newey-West trick from Theory 4. `sm.OLS(y, X)` sets up a least-squares regression. Here X is a column of ones, so the only coefficient is the mean of the VRP. `.fit(cov_type="HAC", cov_kwds={"maxlags": 21})` estimates it with standard errors robust to 21 days of overlap. `tvalues[0]` is the t-statistic of the mean: 2.8.

```
by_reg = v.groupby("regime", observed=True).agg(VIX=("VIX", "mean"), RV_fwd=
("rv21_fwd", "mean"),
                                                VRP=("vrp", "mean"), VRP_pos=
("vrp", lambda x: (x > 0).mean()),
                                                days=("vrp", "size"))
by_reg.round(3).to_csv(OUT / "vrp_by_regime.csv")
```

- `groupby("regime")` splits the days into Low, Normal and High. `observed=True` tells pandas to use only the categories that actually appear, which avoids a warning with categorical columns.

- `.agg(name=(column, function))` is "named aggregation": each argument creates one output column. For example `VRP=("vrp", "mean")` is the mean VRP per regime, and `VRP_pos` uses a small lambda to get the share of positive days. `"size"` counts the rows.
- The result is the VRP-by-regime table (2.1, 1.4 and 4.0 points).

```
mz = sm.OLS(v.rv21_fwd, sm.add_constant(v.VIX)).fit(cov_type="HAC", cov_kwds=
{"maxlags": 21})
vrp_stats.update(mz_alpha=float(mz.params.iloc[0]),
mz_beta=float(mz.params.iloc[1]), mz_r2=float(mz.rsquared))
```

The Mincer-Zarnowitz regression: future RV on VIX. `sm.add_constant` adds the column of ones for the intercept. `params` holds the intercept (1.05) and slope (0.81), and `rsquared` is 0.38. HAC errors are used again because the dependent variable overlaps.

```
RESULTS["vrp"] = {k: round(float(x), 3) for k, x in vrp_stats.items()}
RESULTS["vrp_by_regime"] =
by_reg.round(2).reset_index().to_dict(orient="records")
```

A dictionary comprehension rounds every statistic and stores it. `reset_index()` turns the regime labels back into a normal column, and `to_dict(orient="records")` turns the table into a list of rows, which is how tables are stored in `results.json`.

The chart (fig2_vrp.png). `plt.subplots(2, 1, sharex=True, gridspec_kw={"height_ratios": [2, 1]})` makes two stacked panels sharing a date axis, the top one twice as tall. The top panel plots VIX (blue) and forward RV (orange). The bottom panel uses `fill_between(…, where=v.vrp >= 0)` to shade the VRP blue where VIX was above what followed and red where it was below. The y-axis is cut at −35, so the deepest red (−64 in early March 2020) is clipped. `save(fig, "fig2_vrp.png")` writes it.

5. VIX regimes and Nifty returns (lines 202 to 273)

```
reg_rows = []
for reg, g in df.groupby("regime", observed=True):
    for h, col in [("1d", "fwd1"), ("21d", "fwd21"), ("63d", "fwd63")]:
        x = g[col].dropna()
        lag = {"1d": 1, "21d": 21, "63d": 63}[h]
        m = sm.OLS(x.values, np.ones(len(x))).fit(cov_type="HAC", cov_kwds=
{"maxlags": lag})
        reg_rows.append(dict(regime=reg, horizon=h, n=len(x),
mean_pct=x.mean() * 100, median_pct=x.median() * 100,
                             sd_pct=x.std() * 100, hit_rate=(x > 0).mean(),
nw_t=float(m.tvalues[0]),
                             p5_pct=x.quantile(.05) * 100))
reg_tab = pd.DataFrame(reg_rows)
```

- The outer loop gives each regime's name `reg` and its sub-table `g`. The inner loop goes over three horizons, pairing a label with the column holding that forward return.
- `x` is the forward returns for that regime and horizon, without the NaNs at the end of the sample.
- `lag` looks up the Newey-West lag: 1 for daily, 21 and 63 for the overlapping horizons.
- The OLS-on-a-constant trick again gives the Newey-West t-statistic of the mean.
- Each combination becomes one dictionary of statistics, appended to a list. `pd.DataFrame(reg_rows)` turns the list into the 9-row table saved as `returns_by_regime.csv`. Building a list of dicts and converting once at the end is the standard, fast pattern in pandas.

```
same_day = df.groupby("regime", observed=True).ret.agg(
    mean_pct=lambda x: x.mean() * 100, sd_pct=lambda x: x.std() * 100,
    ann_vol=lambda x: x.std() * math.sqrt(TRADING_DAYS) * 100,
    worst_pct=lambda x: x.min() * 100, days="size")
```

Same-day statistics: each day's own return, grouped by that day's regime. This is where "9% volatility in Low, 13% in Normal, 30% in High" and the −13.9% worst day come from.

```
kw21 = stats.kruskal(*[g.fwd21.dropna() for _, g in df.groupby("regime",
observed=True)])
lev21 = stats.levene(*[g.fwd21.dropna() for _, g in df.groupby("regime",
observed=True)])
kw1 = stats.kruskal(*[g.ret.dropna() for _, g in df.groupby("regime",
observed=True)])
lev1 = stats.levene(*[g.ret.dropna() for _, g in df.groupby("regime",
observed=True)])
```

- The list comprehension builds a list of three series, one per regime. `_` is the Python convention for a value you don't need (the regime name).
- `*` unpacks the list, so `stats.kruskal(*[a, b, c])` is the same as `stats.kruskal(a, b, c)`. Both tests take any number of groups this way.
- `kw1` and `lev1` are the tests quoted in the report (same-day returns: p = 0.66 and p < 0.001). `kw21` and `lev21` use overlapping returns, so their p-values overstate significance.

```
xy = df[["ret", "dvix"]].dropna()
corr = xy.corr().iloc[0, 1]
up, dn = xy[xy.ret > 0], xy[xy.ret < 0]
beta = sm.OLS(xy.dvix, sm.add_constant(xy.ret)).fit()
pred = sm.OLS(df.fwd21.dropna(), sm.add_constant(df.loc[df.fwd21.notna(),
"VIX"])).fit(
    cov_type="HAC", cov_kwds={"maxlags": 21})
```

- `corr`: daily Nifty return against daily log-VIX change, −0.56.
- `up` and `dn` filter rows with boolean masks: `xy[xy.ret > 0]` keeps only up days. Their mean `dvix` gives the asymmetry (−1.9% on up days, +2.3% on down days).
- `beta`: regression of the VIX change on the Nifty return, slope −2.7. Daily changes do not overlap, so plain standard errors are fine here.
- `pred`: next-21-day return on today's VIX. `df.loc[df.fwd21.notna(), "VIX"]` selects VIX on exactly the rows where the forward return exists, so the two sides line up. HAC errors with 21 lags because the returns overlap. Slope 0.18% per VIX point, t = 4.3.

The next lines store all of this in `RESULTS`, rounding each value with a dictionary comprehension.

Figure 1 (Nifty and VIX shaded by regime). The interesting part is the shading:

```
seg = (df.regime != df.regime.shift()).cumsum()
for _, g in df.groupby(seg):
    for a in ax:
        a.axvspan(g.index[0], g.index[-1] + pd.Timedelta(days=1),
color=REGIME_COL[str(g.regime.iloc[0])], lw=0, zorder=0)
```

- `df.regime != df.regime.shift()` is True on each day the regime changes. `.cumsum()` counts the changes so far, which gives every unbroken run of the same regime its own number.
- Grouping by that number gives one group per run. `axvspan` shades the run from its first to its last day (plus one day to close gaps) in that regime's colour, on both panels. `zorder=0` keeps the shading behind the lines.

- The two dashed lines are the 13 and 20 cut-offs. The event labels use `df.VIX.asof(d)`, which returns VIX on date d or the last trading day before it.

Figure 3 (regime returns and the leverage effect).

```
err = 1.96 * t21.mean_pct / t21.nw_t
ax[0].errorbar(range(3), t21.mean_pct, yerr=err.abs(), fmt="none",
ecolor=C["ink"], capsize=4, lw=1)
```

The left panel is a bar chart of mean next-21-day returns by regime. Since t = mean ÷ standard error, the standard error is mean ÷ t, and 1.96 standard errors gives the 95% Newey-West confidence interval drawn as error bars. The right panel is a scatter of daily Nifty returns against VIX changes with the fitted `beta` line drawn through it.

6. The election event study (lines 281 to 383)

```
def straddle_pct(vix_level, days=30):
    T = days / 365; s = vix_level / 100
    return (bs_put(1.0, 1.0, T, s) + bs_call(1.0, 1.0, T, s)) * 100

df["straddle30"] = straddle_pct(df.VIX)
df["implied_daily_move"] = df.VIX / math.sqrt(365) * math.sqrt(365 /
TRADING_DAYS)
df["abs_ret"] = df.ret.abs() * 100
```

- `straddle_pct` prices an ATM put plus an ATM call with spot and strike both set to 1.0. The answer is then automatically a fraction of spot; × 100 makes it a percentage. Because `bs_put` works on arrays, passing the whole VIX column prices a straddle for every day in one call.
- `implied_daily_move` is VIX converted to a one-day standard deviation. Dividing by √365 gives a calendar-day figure; multiplying by √(365/252) converts it to a trading day. The two steps simplify to VIX ÷ √252, written out to show the logic.
- `abs_ret` is the size of each day's move in percent, ignoring direction.

```
win = df.loc["2024-04-01":"2024-07-15"].copy()
exp_jun = pd.Timestamp("2024-06-27")
Kp = 22500.0
Tj = pd.Series(np.clip((exp_jun - win.index).days, 0, None) / 365,
index=win.index)
win["put22500"] = np.where(win.index <= exp_jun, bs_put(win.Close.values, Kp,
Tj.values,

put_iv(win.VIX.values, win.Close.values, Kp)), np.nan)
```

- `df.loc["2024-04-01":"2024-07-15"]` slices rows by date; pandas includes both ends.
- `exp_jun` is the June 2024 monthly expiry, Thursday 27 June. `Kp` is the strike.
- `(exp_jun - win.index).days` is the number of calendar days to expiry for every date in the window. `np.clip(…, 0, None)` turns negative values (dates after expiry) into 0. Dividing by 365 gives T in years.
- `np.where(condition, value, np.nan)` prices the put with Black-Scholes and the skewed IV up to expiry and leaves it blank afterwards. This produces the 409, 545, 217, 903, 335, 112 path.

```
base = df.loc["2024-01-01":"2024-03-31", "VIX"].median()
ev = {}
for d in ["2024-05-31", "2024-06-03"]:
    pre = float(df.VIX.loc[d])
    ev[d] = dict(vix=pre,
                 implied_event_sd_pct=round(math.sqrt(max((pre ** 2 * 30 -
base ** 2 * 29) / 365, 0)), 2),
                 vix_only_daily_sd_pct=round(pre / math.sqrt(TRADING_DAYS),
2))
```

- `base` is the "normal" VIX level, the median for January to March 2024 (14.41).
- For the last close before the exit polls and the last close before the result, the loop computes the event-day standard deviation with the variance formula from the study guide: $\sqrt{((\text{VIX_pre}^2 \times 30 - \text{base}^2 \times 29) \div 365)}$. `max(…, 0)` guards against a negative number under the square root if VIX were below base. Because VIX is in percent, the answer is directly in percent: 5.77 and 4.42.
- `vix_only_daily_sd_pct` is the naive VIX ÷ $\sqrt{252}$ for comparison: 1.55 and 1.32.

```
e = df.loc["2024-05-27":"2024-06-07", ["Close", "ret", "VIX", "straddle30",
"implied_daily_move"]].copy()
e["ret_pct"] = (np.exp(e.ret) - 1) * 100
e["put22500"] = win.put22500.reindex(e.index)
e.round(3).to_csv(OUT / "election_window.csv")
intraday_low = (df.Low.loc["2024-06-04"] / df.Close.loc["2024-06-03"] - 1) *
100
normal = df.loc["2024-01-01":"2024-04-30"]
```

- `e` is the two-week table around the event. `np.exp(ret) - 1` turns log returns into percentage moves (−5.93% on 4 June).
- `reindex(e.index)` copies the put price for exactly those dates.
- `intraday_low` compares the 4 June low with the 3 June close: −8.52%.

- `normal` is January to April 2024, the benchmark period for the medians (VIX 13.8, straddle 3.2%, daily move 0.56%).

The long `RESULTS["election"] = dict(…)` block stores all these numbers, including the day-by-day VIX, straddle and put paths as small dictionaries keyed by date (`str(k.date())` turns a timestamp into text like "2024-06-04").

The cross-event table.

```
for name, d in events.items():
    d = pd.Timestamp(d); i = df.index.get_loc(d)
    pre = df.iloc[i - 1]; row = df.iloc[i]
    rows.append(dict(event=name, date=str(d.date()), nifty_ret_pct=
(math.exp(row.ret) - 1) * 100,
                     vix_prev=pre.VIX, vix=row.VIX, vix_chg_pct=(row.VIX /
pre.VIX - 1) * 100,
                     straddle_prev_pct=pre.straddle30,
straddle_pct=row.straddle30,
                     move_vs_implied=abs(row.ret) * 100 /
pre.implied_daily_move))
```

- `df.index.get_loc(d)` finds the row number of the event date, so `iloc[i - 1]` is the previous trading day, whatever the calendar gap.
- Each event gets the Nifty move, VIX before and after, straddle before and after, and the move divided by the previous day's implied daily move.

```
nd = df[(df.abs_ret < df.abs_ret.quantile(.9))]
```

The "normal day" row uses every day except the biggest 10% of moves, and reports medians. `nd.implied_daily_move.shift(1)` uses the previous day's implied move, consistent with the event rows.

```
top = df.dvix.nlargest(20).index
```

`nlargest(20)` picks the 20 days with the biggest rise in log VIX; `.index` gives their dates. The block then averages the Nifty return, the VIX jump and the jump in the straddle (`df.straddle30 - df.straddle30.shift()`) on those days, and the average absolute move on all other days (`df.drop(top)`).

Figures 4 and 5. Figure 4 stacks Nifty, VIX (with the January–April median as a dashed line) and the straddle, with 4 June shaded using `axvspan` from 12 hours before to 12 hours after. Figure 5 plots the 22,500 put's modelled premium; `mdates.DateFormatter("%d-%b")` formats the dates as "04-Jun".

7. The backtest engine `run()`, line by line (lines 405 to 508)

This is the heart of the project. One function simulates any of the strategies, day by day, and returns the daily portfolio value plus a ledger of every option trade.

Setting up the calendar

```
bt = df.loc[:, ["Close", "VIX", "regime"]].copy()
dates = bt.index
first_of_month = pd.Series(dates, index=dates).groupby([dates.year,
dates.month]).transform("min")
roll_dates = set(first_of_month.unique())
roll_list = sorted(roll_dates)
```

- `bt` is the backtest's input: just Nifty close, VIX and regime for each day.
- `first_of_month` groups the dates by (year, month) and, with `transform("min")`, gives every date the earliest date in its month. `unique()` leaves one date per month: the first trading day. Those are the roll dates.
- `roll_dates` is a `set`, which makes "is today a roll date?" an instant check. `roll_list` is the same dates sorted, so they can be counted forwards.

```
def next_roll(d, k):
    i = roll_list.index(d)
    return roll_list[i + k] if i + k < len(roll_list) else None
```

The expiry date of an option bought on roll date `d` with a `k`-month tenor: the roll date `k` positions later. If that is beyond the end of the data it returns `None`, and no option is opened.

The function signature

```
def run(strategy, low=LOW_VIX, spike1=SPIKE_1, spike2=SPIKE_2, tenor=TENOR,
moneyness=PUT_MONEYNESS,
        skew=SKEW_PER_PCT, half_spread=HALF_SPREAD, tranche=1.0,
stagger=False, trail=TRAIL):
```

`strategy` is a text code: `"unhedged"`, `"always_put"`, `"always_spread"`, `"timed_hold"`, `"timed_all"`, `"timed_ladder"`, `"timed_trail"` or `"timed_hybrid"`. Every other argument defaults to the constants at the top, so `run("timed_hybrid")` is the proposal and the robustness section changes one argument at a time, for example `run("timed_hybrid", tenor=2)`. `tranche` is the fraction of the portfolio each new put covers, and `stagger=True` keeps old puts instead of replacing them.

The state the simulation carries

```
    units = 1.0 / bt.Close.iloc[0]   # start with 1.0 of wealth in Nifty
    pos = []                          # dicts: K, exp, qty, sign(+1 long/-1
short), trade_id
    trades = []                       # hedge cashflow ledger
    rec = []
    tid = 0
```

- `units`: how many "units" of the index the portfolio owns. Starting wealth is 1.0, so units = 1 ÷ the first close. Portfolio equity on any day is units × close.
- `pos`: the list of open options. `trades`: the ledger. `rec`: one record per day. `tid`: a counter that numbers each new option.

Four helper functions inside `run()`

They are defined inside `run()` so they can see and change its variables.

```
    def iv(vixl, S, K, side):
        otm = max(0.0, (1 - K / S) * 100)
        return max((vixl + skew * otm + side * half_spread) / 100, 0.01)
```

The backtest's implied volatility: VIX + skew × percent OTM, plus `side × half_spread`. `side` is +1 when buying (pay the offer), −1 when selling (hit the bid) and 0 for a mid-market valuation. The floor of 1% stops a negative volatility.

```
    def price(p, d, S, vixl, side=0):
        T = (p["exp"] - d).days / 365
        if T <= 0:
            return max(p["K"] - S, 0.0)
        return float(bs_put(S, p["K"], T, iv(vixl, S, p["K"], side)))
```

The value of position `p` on date `d`. Time to expiry is in calendar days ÷ 365. On or after expiry it is the intrinsic value; otherwise Black-Scholes with the IV above.

```
    def close_pos(p, d, S, vixl, frac, reason, regime):
        nonlocal units
        q = p["qty"] * frac
        px = price(p, d, S, vixl, side=-1 if p["sign"] > 0 else +1)
        cash = p["sign"] * q * px
        units += cash / S
        p["qty"] -= q
        trades.append(dict(date=d, trade_id=p["tid"], action=reason,
K=p["K"], qty=q * p["sign"], price=px,
                           cash=cash, vix=vixl, regime=regime,
open_regime=p["open_regime"]))
```

Sells (or buys back) a fraction `frac` of a position.

- `nonlocal units` lets this inner function change the outer `units` variable.
- `q` is the quantity being closed: all of it with `frac=1.0`, half with 0.5.
- Price: a long put is sold at the bid (side −1); a short put is bought back at the offer (+1).
- `cash` is positive when selling a long put. `units += cash / S` buys Nifty with the proceeds the same day.
- The position's quantity is reduced, and the trade is written to the ledger with the reason ("expiry", "roll_sell", "monetise_1", "monetise_trail" and so on), the regime today and the regime when it was opened.

```
    def open_pos(d, S, vixl, K, months, sign, regime, frac=1.0):
        nonlocal units, tid
        exp = next_roll(d, months)
        if exp is None:
            return
        V = units * S + sum(price(p, d, S, vixl) * p["qty"] * p["sign"] for p
in pos)
        qty = frac * V / S
        p = dict(K=K, exp=exp, qty=qty, sign=sign, tid=tid,
open_regime=regime, sold1=False)
        px = price(p, d, S, vixl, side=+1 if sign > 0 else -1)
        cash = -sign * qty * px
        units += cash / S
        pos.append(p)
        trades.append(dict(date=d, trade_id=tid, action="open", K=K, qty=qty
* sign, price=px, cash=cash,
                           vix=vixl, regime=regime, open_regime=regime))
        tid += 1
```

Buys (sign +1) or sells (sign −1) a new put.

- The expiry comes from `next_roll`. If the data ends first, nothing is opened.

- `V` is today's total portfolio value: Nifty holdings plus any options still open, valued at mid.
- `qty = frac × V ÷ S` sizes the put so it covers `frac` of the whole portfolio (100% by default). Quantity is in index units, so a put on qty units pays qty × (K − S) at expiry.
- The new position records its strike, expiry, size, direction, id, the regime it was opened in, and `sold1=False` (the first half has not been monetised yet).
- Buying pays the offer. `cash` is negative, so `units += cash / S` sells Nifty units to pay the premium. Selling (the short leg of the spread) receives the bid and buys units.

The daily loop

```
    for d, row in bt.iterrows():
        S, vixl, regime = row.Close, row.VIX, str(row.regime)
```

`iterrows()` walks through the days in order. `S`, `vixl` and `regime` are today's close, VIX close and regime label.

Step 1: settle expiries.

```
        for p in pos:
            if p["qty"] > 0 and p["exp"] <= d:
                close_pos(p, d, S, vixl, 1.0, "expiry", regime)
        pos = [p for p in pos if p["qty"] > 1e-12]
        is_roll = d in roll_dates
```

Any option whose expiry has arrived is closed at intrinsic value (`price` returns max(K − S, 0) when T ⩽ 0). The list comprehension then drops positions with nothing left; `1e-12` avoids keeping tiny floating-point leftovers. Expiries are always roll dates, so on a roll date the old 1-month put settles first and the new one is bought after.

Step 2a: always-on strategies.

```
        if strategy == "always_put" and is_roll:
            open_pos(d, S, vixl, round(S * moneyness / 50) * 50, 1, +1,
regime)
        elif strategy == "always_spread" and is_roll:
            open_pos(d, S, vixl, round(S * moneyness / 50) * 50, 1, +1,
regime)
            open_pos(d, S, vixl, round(S * SPREAD_SHORT / 50) * 50, 1, -1,
regime)
```

On every roll date: buy a 1-month put at 95% of spot, rounded to the nearest 50-point strike. The spread also sells a 1-month 85% put.

Step 2b: timed strategies, monetisation first.

```
        elif strategy.startswith("timed"):
            monetise = strategy in ("timed_ladder", "timed_all",
"timed_trail", "timed_hybrid")
            if monetise:
                for p in pos:
                    if strategy in ("timed_trail", "timed_hybrid"):
                        if vixl >= spike1:
                            p["peak"] = max(p.get("peak", 0.0), vixl)
                        if strategy == "timed_hybrid" and not p["sold1"] and
vixl >= spike1:
                            close_pos(p, d, S, vixl, 0.5, "monetise_1",
regime); p["sold1"] = True
                        if p.get("peak") and vixl <= (1 - trail) * p["peak"]
and p["qty"] > 0:
                            close_pos(p, d, S, vixl, 1.0, "monetise_trail",
regime)
                    elif strategy == "timed_ladder":
                        if not p["sold1"] and vixl >= spike1:
                            close_pos(p, d, S, vixl, 0.5, "monetise_1",
regime); p["sold1"] = True
                        if p["sold1"] and vixl >= spike2 and p["qty"] > 0:
                            close_pos(p, d, S, vixl, 1.0, "monetise_2",
regime)
                    elif vixl >= spike1:
                        close_pos(p, d, S, vixl, 1.0, "monetise", regime)
                pos = [p for p in pos if p["qty"] > 1e-12]
```

`timed_hold` skips this block entirely: it never sells early. For the others, each open put is checked against today's VIX close.

- **Trailing and hybrid.** Once VIX closes at or above 22, the put is "armed" and remembers its `peak`, the highest VIX close at or above 22 so far. `p.get("peak", 0.0)` returns 0 if no peak is stored yet. The hybrid also sells half on the first such day and sets `sold1` so it never does that again. Then, on any later day when VIX has fallen to 80% of the peak or below, the rest is sold. `p.get("peak")` is `None` until armed, so the fade check cannot fire before a spike. It also cannot fire on the arming day, since the peak is then at least today's VIX. `timed_trail` is the same without the half sale.
- **Ladder.** Half at 22, the rest at 30. If VIX never reaches 30, the second half is held to expiry.
- **Sell all** (`timed_all`) falls to the last `elif`: everything at 22.

Step 2c: timed strategies, buy or roll.

```
            if is_roll and vixl < low:
                if not stagger:
                    for p in pos:
                        close_pos(p, d, S, vixl, 1.0, "roll_sell", regime)
                    pos = []
                open_pos(d, S, vixl, round(S * moneyness / 50) * 50, tenor,
+1, regime, frac=tranche)
```

Only on a roll date with VIX below the threshold. In the normal version the old put (or whatever half of it is left) is sold at the bid and a fresh `tenor`-month 95% put is bought for the whole portfolio. If VIX is 13 or above, nothing happens and the existing put is simply held. In the staggered version old puts are kept and a new tranche (a third or a sixth of the portfolio) is added. Note the trick that defines the "Always 3M" baseline: `run("timed_hold", low=1e9)`. With a threshold of a billion, VIX is always "low", so it rolls every month.

Step 3: record the day.

```
        hedge_mtm = sum(price(p, d, S, vixl) * p["qty"] * p["sign"] for p in
pos)
        rec.append(dict(date=d, equity=units * S, hedge=hedge_mtm,
value=units * S + hedge_mtm,
                        notional_hedged=sum(p["qty"] * p["sign"] for p in pos
if p["sign"] > 0) * S))
    out = pd.DataFrame(rec).set_index("date")
    return out, pd.DataFrame(trades)
```

The options are valued at mid, the day's total value is equity plus options, and the hedged notional is recorded (used later for "time hedged"). After the last day, the records become a table indexed by date, returned with the ledger.

One trade traced through the code

The proposal's first trade, from `ledger_s5_timed_hybrid.csv`:

1. **2 January 2020**, a roll date, VIX 11.49 < 13. `open_pos` buys a put with strike 11,650 (95% of spot, rounded) expiring on `next_roll(d, 3)` = 1 April 2020. Quantity 0.0000814 units = the whole portfolio ÷ spot. Price 92.86 points, cash −0.0076, so 0.76% of the portfolio is sold to pay for it.
2. **3 February 2020**, a roll date, but VIX is above 13, so nothing happens. The put is held.
3. **28 February 2020**, VIX closes at 23.24 ⩾ 22. The put is armed with peak 23.24, and `close_pos(…, 0.5, "monetise_1")` sells half at the bid for 545.84 points. Cash +0.0222 buys Nifty units.

4. **March 2020**, VIX keeps rising, so the peak keeps rising (to the 83.6 close on 24 March). The fade rule needs VIX at or below 0.8 × peak, about 67.
5. **31 March 2020**, VIX closes at 64.41, more than 20% below its peak. `close_pos(…, 1.0, "monetise_trail")` sells the remaining half for 3,050.48 points, cash +0.1242.

The trade cost 0.76% of the starting portfolio and brought back 14.6% of it. That single trade is most of the proposal's advantage over the whole six years.

8. Metrics, running the strategies, cost by regime and episodes (lines 511 to 648)

The metrics function

```
def metrics(val, rf=R):
    r = val.pct_change().dropna()
    yrs = (val.index[-1] - val.index[0]).days / 365.25
    cagr = (val.iloc[-1] / val.iloc[0]) ** (1 / yrs) - 1
    vol = r.std() * math.sqrt(TRADING_DAYS)
    dd = val / val.cummax() - 1
    downside = r[r < 0].std() * math.sqrt(TRADING_DAYS)
    return dict(final=val.iloc[-1], CAGR=cagr * 100, vol=vol * 100, sharpe=
(cagr - rf) / vol,
                sortino=(cagr - rf) / downside, maxDD=dd.min() * 100,
calmar=cagr / abs(dd.min()),
                worst_day=r.min() * 100, worst_21d=(val / val.shift(21) -
1).min() * 100)
```

Takes a series of daily portfolio values and returns the performance numbers defined in the study guide.

- `pct_change()` gives daily simple returns. `yrs` is the length in years, using 365.25 days to allow for leap years.
- `cagr` is (end ÷ start) to the power 1 ÷ years, minus 1.
- `val.cummax()` is the running highest value so far. `val / val.cummax() - 1` is the drawdown on each day (0 at a new high, negative below it). Its minimum is the maximum drawdown.
- `downside` is the standard deviation of only the negative daily returns, annualised, for the Sortino ratio.
- `val / val.shift(21) - 1` is the return over every 21-day window; its minimum is the worst month.

Running all seven strategies

```
PROPOSED = "VIX-timed 3M put + hybrid monetisation (proposed)"
STRATS = {"Unhedged": ("unhedged", {}), "Always 1M 95% put": ("always_put",
{}),
          "Always 1M 95/85 put spread": ("always_spread", {}),
          "Always 3M 95% put, rolled monthly": ("timed_hold", {"low": 1e9}),
          "VIX-timed 3M put, hold": ("timed_hold", {}),
          PROPOSED: ("timed_hybrid", {}),
          "VIX-timed 3M put, sell all at VIX 22": ("timed_all", {})}
SLUG = {n: f"s{i}_{s}" for i, (n, (s, _)) in enumerate(STRATS.items())}
runs, ledgers = {}, {}
for name, (s, kw) in STRATS.items():
    runs[name], ledgers[name] = run(s, **kw)
vals = pd.DataFrame({k: r.value for k, r in runs.items()})
```

- `STRATS` maps each display name to the strategy code and any extra arguments. "Unhedged" works because `run` with an unknown strategy code simply never trades.
- `SLUG` builds short file names (`s5_timed_hybrid` and so on) from each strategy's position and code.
- `run(s, **kw)` unpacks the dictionary of extra arguments, so `{"low": 1e9}` becomes `run("timed_hold", low=1e9)`.
- `vals` is one table with a column per strategy and a row per day: the data behind the equity chart, saved as `backtest_values.csv`.

```
perf = pd.DataFrame({k: metrics(vals[k]) for k in vals}).T
for name, led in ledgers.items():
    if len(led) == 0:
        perf.loc[name, ["premium_paid_pa", "recovered_pa",
"net_hedge_cost_pa", "share_time_hedged"]] = 0
        continue
    led.to_csv(OUT / f"ledger_{SLUG[name]}.csv", index=False)
    pv = vals[name].reindex(led.date).values
    led["cash_pct"] = led.cash.values / pv * 100
    paid = -led.loc[led.cash < 0, "cash_pct"].sum()
    rec_ = led.loc[led.cash > 0, "cash_pct"].sum()
    perf.loc[name, "premium_paid_pa"] = paid / yrs
    perf.loc[name, "recovered_pa"] = rec_ / yrs
    perf.loc[name, "net_hedge_cost_pa"] = (paid - rec_) / yrs
    perf.loc[name, "share_time_hedged"] = (runs[name].notional_hedged >
0).mean() * 100
```

- The first line applies `metrics` to every column; `.T` transposes so strategies are rows.

- For each strategy with trades, the ledger is saved, then every cash flow is divided by the portfolio value on that date (`reindex(led.date)` looks the values up) to express it as a % of the portfolio.
- `paid` sums the negative cash flows (premium spent) and flips the sign; `rec_` sums the positive ones. Both are divided by the number of years for per-year figures.
- `share_time_hedged` is the share of days with any long put open.

Cost and payoff by regime

```
for name in list(STRATS)[1:]:
    led = ledgers[name].copy()
    pv = vals[name].reindex(led.date).values
    led["cash_pct"] = led.cash.values / pv * 100
    for basis in ("open_regime", "regime"):
        g = led.groupby(basis)
        for reg in ["Low", "Normal", "High"]:
            if reg not in g.groups:
                regime_rows.append(dict(strategy=name, basis=basis,
regime=reg, paid=0.0, received=0.0, net=0.0, n_trades=0))
                continue
            x = g.get_group(reg)
            paid = -x.loc[x.cash < 0, "cash_pct"].sum(); recv = x.loc[x.cash
> 0, "cash_pct"].sum()
            regime_rows.append(dict(strategy=name, basis=basis, regime=reg,
paid=paid, received=recv, net=recv - paid,
                                    n_trades=int((x.action ==
"open").sum())))
```

- `list(STRATS)[1:]` skips Unhedged.
- Two ways of grouping the ledger. `open_regime` gives each cash flow to the regime the option was **bought** in, so an option's whole life lands in one bucket. `regime` gives each cash flow to the regime on the day it **happened**.
- If a strategy never traded in a regime (the timed ones never buy in Normal or High), a row of zeros is written so the table stays complete. Otherwise, premium paid, money received and the net are summed, and `n_trades` counts the "open" rows.
- The totals are cumulative over the 6.28 years. The report and chart divide by years to get per-year figures.

```
dr = vals.pct_change()
bt["regime_prev"] = bt.regime.shift()
excess = dr.sub(dr["Unhedged"], axis=0).join(bt.regime_prev)
regime_excess = (excess.groupby("regime_prev", observed=True).sum() *
100).drop(columns="Unhedged")
days_in = excess.groupby("regime_prev", observed=True).size()
regime_excess_pa = regime_excess.div(days_in, axis=0) * TRADING_DAYS
```

The day-by-day view.

- `dr` holds every strategy's daily return. `dr.sub(dr["Unhedged"], axis=0)` subtracts the unhedged return from every column, row by row: the hedge's contribution each day.
- `regime_prev` is yesterday's regime, the one known when the day starts, so the grouping uses no future information.
- Summing by regime gives the cumulative contribution in each regime; dividing by the number of days in that regime and multiplying by 252 annualises it. This is where "−2.1% a year on low-VIX days, +2.0% on normal days, +6.3% on high-VIX days" comes from.

Stress episodes

```
ep = pd.DataFrame({k: (vals.loc[b] / vals.loc[a] - 1) * 100 for k, (a, b) in
                   ((k, (vals.index[vals.index.get_indexer([pd.Timestamp(a)],
method="bfill")[0]],
                         vals.index[vals.index.get_indexer([pd.Timestamp(b)],
method="ffill")[0]])) for k, (a, b) in episodes.items())}).T
```

The densest line in the script. Read it from the inside out.

- `episodes` maps each name to a (start, end) pair of dates.
- `get_indexer([date], method="bfill")` finds the first trading day on or after the start date; `method="ffill"` finds the last trading day on or before the end date. This snaps calendar dates to real trading days.
- The inner generator produces (name, (start_day, end_day)) pairs. The outer dictionary comprehension computes each strategy's return from start to end: `vals.loc[b] / vals.loc[a] - 1` divides two rows, giving one return per strategy.
- `.T` makes episodes the rows.

Trade summary of the proposal

```
L = ledgers[PROPOSED]
tr = L.groupby("trade_id").agg(open_date=("date", "first"), open_vix=("vix",
"first"), cash=("cash", "sum"),
                               exit=("action", lambda a:
",".join(sorted(set(a) - {"open"}))))
```

Groups the ledger by option: the date and VIX when bought, the net cash over its life, and how it ended (the set of its actions minus "open", joined into text such as "monetise_1,monetise_trail"). From this come the 22 trades, 5 winners, 3 monetised, and the list of monetisation sales.

Charts

- **Figure 6** plots every strategy's value (top) and drawdown (bottom), with the proposal and unhedged drawn thicker (`lw=2.0`).
- **Figure 7** is a grouped bar chart of net option P&L per year by the regime at purchase. With six strategies per group, `xx + (i − 2.5) × w` offsets bar i so the six bars sit side by side, centred on each regime's tick.

9. Robustness grids, the heatmap and saving results (lines 654 to 725)

Every robustness test is the same pattern: loop over values of one or two settings, call `run()` with them, compute `metrics()` , collect a row.

```
grid = []
for low in [12, 13, 14, 15, 16, 1e9]:
    for s1 in [20, 22, 25, 30]:
        o, _ = run("timed_hybrid", low=low, spike1=s1)
        m = metrics(o.value)
        grid.append(dict(buy_below=str(int(low)) if low < 100 else "always",
spike1=s1, CAGR=m["CAGR"], maxDD=m["maxDD"],
                         sharpe=m["sharpe"], calmar=m["calmar"]))
```

- 6 buy thresholds × 4 first-sale triggers = 24 backtests of the proposal. `1e9` means "always buy" and is labelled that way.
- `o, _ = run(…)` keeps the daily values and throws away the ledger (`_`).
- Saved as `sensitivity_thresholds.csv` and drawn as the heatmap in Figure 8.

```
for sk in [0.0, 0.6, 1.0]:
    for hs in [0.0, 0.5, 1.0]:
        for strat in ["always_put", "timed_hybrid"]:
            o, _ = run(strat, skew=sk, half_spread=hs)
```

Pricing stress test: 3 skew levels × 3 spreads × 2 strategies = 18 runs. Comparing the always-on put with the proposal under the same assumptions shows whether the timing advantage survives costlier options. It does: the proposal returned 12.9% to 15.1%, the always-on put 6.0% to 11.1%.

```
for ten in [1, 2, 3, 6]:
    o, _ = run("timed_hybrid", tenor=ten)
for mny in [0.90, 0.95, 1.00]:
    o, _ = run("timed_hybrid", moneyness=mny)
```

Tenor (1, 2, 3, 6 months) and strike (10% OTM, 5% OTM, ATM), one at a time. These rows are appended to the same table, `sensitivity_pricing.csv`.

```
RULES = {"Hold to expiry": "timed_hold", "Sell all at VIX 22": "timed_all",
"Ladder 22 / 30": "timed_ladder",
         "Trailing (fade 20%)": "timed_trail", "Hybrid (proposed)":
"timed_hybrid"}
for ten, tr, st, lab in [(2, 1, False, "2M rolled"), (3, 1, False, "3M
rolled"), (3, 1 / 3, True, "3M staggered thirds"),
                         (6, 1 / 6, True, "6M staggered sixths")]:
    for rn, rs in RULES.items():
        o, _ = run(rs, tenor=ten, tranche=tr, stagger=st)
        m, mp = metrics(o.value), metrics(o.value.loc["2020-07-01":])
        mat.append(dict(structure=lab, rule=rn, CAGR=m["CAGR"],
maxDD=m["maxDD"], sharpe=m["sharpe"],
                        covid=(o.value.asof(pd.Timestamp("2020-03-23")) /
o.value.asof(pd.Timestamp("2020-01-17")) - 1) * 100,
                        CAGR_ex_covid=mp["CAGR"],
maxDD_ex_covid=mp["maxDD"]))
```

The most revealing test: 4 structures × 5 monetisation rules = 20 runs.

- Each structure is a tuple (tenor, tranche size, stagger flag, label). "3M staggered thirds" buys a 3-month put on a third of the portfolio each eligible month and keeps the older ones, so up to three overlap.
- Each run is measured twice: over the full period (`m`) and from 1 July 2020 (`mp`), which drops the COVID crash.
- `covid` is the return from 17 January to 23 March 2020, read with `.asof()`.

- This table is where you can see that the full-period ranking depends on COVID, while the post-COVID CAGRs of all 20 runs sit between 13.8% and 15.3% a year.

```
post = vals.loc["2020-07-01":]
RESULTS["ex_covid_performance"] = {k: {kk: round(float(vv), 3) for kk, vv in
metrics(post[k]).items()} for k in post}
```

The main seven strategies re-measured from July 2020, a nested dictionary comprehension: for each strategy `k`, a dictionary of its rounded metrics. This gives the "CAGR from July 2020" column (15.4% unhedged against 14.9% for the proposal).

Figure 8, the heatmap.

```
h = grid.pivot(index="buy_below", columns="spike1", values=col)
h = h.reindex(sorted(h.index, key=lambda x: 1e9 if x == "always" else
float(x)))
good = h.values
norm_ = (good - good.min()) / (np.ptp(good) + 1e-9)
a.imshow(norm_, cmap="Blues", aspect="auto", vmin=-0.15, vmax=1.15)
```

- `pivot` reshapes the long grid into a matrix: buy thresholds down the side, triggers across the top.
- The thresholds are text ("12" … "always"), so they are sorted with a key that puts "always" last.
- `norm_` rescales the values to 0–1 (`np.ptp` is max minus min; `1e-9` avoids dividing by zero). Higher CAGR and shallower drawdown (less negative) both give larger values, so darker always means better. An earlier version coloured drawdowns the wrong way round; `good = h.values` is the fix.
- `imshow` draws the colour grid. `vmin` / `vmax` slightly beyond 0–1 keep the extremes from being pure white or black. The loop after it writes each number in its cell, white on dark cells and black on light ones.

Saving everything.

```
with open(OUT / "results.json", "w") as f:
    json.dump(RESULTS, f, indent=1, default=str)
print("done")
```

`json.dump` writes the `RESULTS` dictionary to a file. `default=str` converts anything JSON cannot store directly (like dates) to text. Every number in the report and the deck can be traced to a key in this file.

Deck walkthrough: fdrm_vix_presentation.pptx

The deck has 17 slides in five sections: overview (1–3), volatility and India VIX (4–7), the election (8–9), strategy and backtest (10–15), and limitations and conclusion (16–17). Every slide has speaker notes in the PowerPoint file. For each slide below: what it shows, where every number comes from (so you can defend it), and what to say. At about a minute per slide the talk runs 15 to 18 minutes; if you are short of time, slides 5, 9 and 13 are the easiest to compress.

Slides 1 to 9

Slide 1: title

- **Shows:** "Nifty Options, India VIX and a VIX-Timed Hedge" on a dark background, with a one-line subtitle.
- **Say:** "I'll cover the three syllabus topics (implied versus realised volatility, how India VIX is built and used, and VIX regimes against Nifty returns), then the 4 June 2024 election, and finally a protection strategy I designed and backtested."

Slide 2: four numbers that sum up the study

- **Shows:** four stat cards and a one-line takeaway.
- **Sources:** 2.2 pts and 78% are `vrp.mean_vrp` and `vrp.share_positive` in `results.json`. 9% → 30% is `sameday_by_regime` ann_vol for Low and High. 4.2× is the 22,500 put going from 216.5 (3 June) to 902.5 (4 June) in `election.put22500_path`. −22% vs −38% is maxDD of the proposal and Unhedged in `backtest_performance.csv`.
- **Say:** "If you remember four numbers: options are overpriced by about 2 vol points, fear regimes triple volatility, election-day puts jumped four-fold, and the timed hedge cut the worst fall from 38% to 22%. The idea in one line: buy protection when VIX is calm and sell it into the spike."
- **Be ready for:** "Is the −22% just COVID?" Yes, mostly; slide 15 deals with it.

Slide 3: real prices, modelled option premiums

- **Shows:** two cards (real data, modelled premiums) and the three regime bands with their share of days.

- **Sources:** the data files and the assumptions block at the top of the script. Shares of days (27%, 48%, 25%) are `sample.regime_share`.
- **Say:** "Index levels are real daily closes, and I spot-checked election day against NSE. Historical option chains weren't available, so premiums are Black-Scholes prices using India VIX plus a put skew and a bid/ask haircut. I'll show later that the conclusions survive changing those assumptions. Throughout, Low means VIX below 13, High means above 20."
- **Be ready for:** "Why not real option prices?" See the Q&A in the Study guide tab.

Slide 4: VIX usually overstates future volatility

- **Shows:** Figure 2 (VIX against the next 21 days' realised volatility, with the VRP shaded below) and a bar chart of VRP by regime.
- **Sources:** `fig2_vrp.png`; bars from `vrp_by_regime.csv` (2.1, 1.4, 4.0); t = 2.8 is `vrp.nw_t_mean_vrp`; correlations 0.86 and 0.61 are `vrp.corr_vix_rvtrail` and `vrp.corr_vix_rvfwd`.
- **Say:** "Blue is India VIX, orange is the volatility Nifty actually delivered over the following month. VIX was higher 78% of the time, by 2.2 points on average, and the gap is statistically significant. It is biggest right after shocks, 4 points in the high regime, which tells a hedger when not to buy. The red patches are the exceptions: crashes, when realised volatility blew through what was priced."

Slide 5: how India VIX is built

- **Shows:** NSE's four steps, the variance formula, and the 15.0 and 16.1 replication results.
- **Sources:** `vix_construction.flat15` and `vix_construction.skewed_atm15`.
- **Say:** "India VIX takes near and next-month Nifty options, computes a model-free variance for each expiry, interpolates to 30 days and scales it to an index. The $1/K^2$ weight means low-strike puts count a lot. I re-implemented the formula: with every option at 15% it returns exactly 15, which checks the code; add put skew and it returns 16.1. So VIX is part volatility forecast, part price of crash insurance."

Slide 6: India VIX as a fear gauge, three traits

- **Shows:** Figure 1 (Nifty and VIX shaded by regime) and three traits.
- **Sources:** `fig1_nifty_vix_regimes.png`; +2.3% and −1.9% are `regimes.mean_dlogvix_on_down_days` and `_up_days`; −0.56 is `regimes.corr_ret_dlogvix`.

- **Say:** "Three things make VIX a fear gauge. It rises when the market falls, by about 2.7% for each 1% fall in Nifty. Its regimes persist for weeks, but every spike came back to the 12 to 16 range. And it moves ahead of known events: before the 2024 result it climbed for six weeks while Nifty went nowhere."

Slide 7: VIX regimes separate risk, not return

- **Shows:** two bar charts (volatility by regime; next-21-day return by regime) and three bullets.
- **Sources:** `sameday_by_regime.csv` (9.0, 13.2, 30.1); `returns_by_regime.csv` 21d means (1.1, −0.2, 3.4); Levene and Kruskal p-values from `regimes`; 0.18% and t = 4.3 are `regimes.fwd21_on_vix_slope_pct_per_pt` and `fwd21_on_vix_t`; −14.6% is the Normal 63d `p5_pct`.
- **Say:** "Volatility triples from the low to the high regime, and that difference is unmistakable. Average same-day returns don't differ significantly. High VIX was actually followed by strong rebounds, but that is mostly the 2020 recovery, so I don't treat it as a timing signal. The important detail for hedging: the worst three-month outcomes started in the normal regime, while VIX was rising from calm. Protection has to be in place before that happens."
- **Be ready for:** "Why Newey-West?" Overlapping returns; see Theory 4.

Slide 8: 4 June 2024, build-up, spike, collapse

- **Shows:** Figure 4 (Nifty, VIX and the 30-day straddle, April to July 2024) and the four phases.
- **Sources:** `election.vix_min_apr_2024` (10.2), `election.vix_path`, `election.straddle30_path`, `election.nifty_ret_*`, `election.intraday_low_4jun_pct` (−8.52), `election.vix_change_5jun_pct` (−29.4).
- **Say:** "This is what a scheduled event looks like in option prices. VIX more than doubled over six weeks while Nifty was flat. The exit polls on 3 June made protection cheapest the day before it was needed. On result day Nifty fell 5.9%, and the June 22,500 put went from 217 to 903 points. A day later VIX was down 29%: the premium vanished even though Nifty was still below where it started."

Slide 9: priced the size, not the direction

- **Shows:** the cross-event table and three stat cards.
- **Sources:** `event_comparison.csv`; 5.8% is `election.events.2024-05-31.implied_event_sd_pct`; 14.4 is `election.base_vix_q1_2024`.

- **Say:** "If the extra variance in the pre-election VIX belongs to the one event day, it implies a 5.8% move. The actual move was 5.9%. So the options market sized the event well; it didn't know the direction. Compared with other shocks, the election is unusual because VIX was already high going in. The unscheduled shocks of 2024 and 2025 hit when VIX was near 14, which is exactly when a calm-market hedge is already in place."
- **Be ready for:** "Why 4.6× if the market priced it well?" The naive measure spreads event risk across the month; the event-variance method concentrates it on the day.

Slides 10 to 17

Slide 10: proposed hedge, buy in calm, sell into fear

- **Shows:** the five rules as numbered cards (buy, roll, wait, sell half, sell the rest), with three reasons underneath.
- **Sources:** the rules are the `timed_hybrid` branch of `run()`. 11.7 and 9.5% are Low-regime VIX and forward RV in `vrp_by_regime.csv`; 4.0 pts is the High-regime VRP.
- **Say:** "The rule is mechanical. On the first trading day of each month, if VIX is below 13, buy a 3-month put 5% below spot on the whole portfolio, and keep rolling it while VIX stays low. If VIX is 13 or higher, buy nothing new. When VIX first closes at 22, sell half; when the spike fades 20% from its peak, sell the rest and put the money back into Nifty. Three months because it decays more slowly and survives a gap between the calm signal and the crash."
- **Example to tell** (from the notes): "The January 2020 put cost 93 points. Half was sold in late February for 546, the rest on 31 March for 3,050."

Slide 11: backtest, higher return, smaller falls

- **Shows:** Figure 6 (growth of 1 and drawdowns for all seven strategies) and four stat cards.
- **Sources:** `backtest_performance.csv`: CAGR 13.9 vs 11.1, maxDD −22.0 vs −38.4, premium_paid_pa 2.9 vs 7.5, share_time_hedged 45.
- **Say:** "The thick blue line is the proposal, black is plain Nifty. Over 2020 to 2026 it returned 13.9% a year against 11.1%, with a worst fall of 22% instead of 38%. It held puts only 45% of the time and spent 2.9% a year on premium, against 7.5% for buying a put every month. Notice where the gap opens: March 2020. After that the lines move almost in parallel, and I'll come back to what that means."

Slide 12: always-on hedges cost more than they saved

- **Shows:** two horizontal bar charts (annual return; maximum drawdown) for all seven strategies, the proposal in blue.
- **Sources:** CAGR and maxDD columns of `backtest_performance.csv`; net cost range from `net_hedge_cost_pa` (3.0 to 4.9 for always-on, −4.4 to +0.2 for timed).
- **Say:** "Every always-on hedge reduced drawdowns, but at 3 to 5% of the portfolio a year they lost more return than they saved. 'Timed, hold' looks best, but only because it held the whole January 2020 put through the crash. The proposal sells half early, which gives up some of that but protects against spikes that reverse quickly."
- **Be ready for:** "Then why not propose hold?" See the Q&A in the Study guide tab.

Slide 13: hedges bought in calm markets paid

- **Shows:** a clustered bar chart of net option P&L per year by the regime at purchase, and four stat cards for the day-by-day view.
- **Sources:** chart values are `hedge_cost_payoff_by_regime.csv` rows with basis `open_regime`, `net` ÷ 6.28 years. Cards are `excess_return_by_regime_annualised.csv` (−2.14, +1.99, +6.32 for the proposal; −9.15 for the always-on put in High).
- **Say:** "Here every option's whole life is credited to the regime when it was bought. Every always-on hedge lost money in every regime. The proposal only ever buys in the low regime, and those purchases made 2.2% of the portfolio a year. On the right, day by day: the proposal costs about 2% a year on calm days, the insurance premium, and earns it back on normal and especially high-VIX days. The always-on put lost 9% a year on high-VIX days because it kept buying expensive protection just before rebounds."

Slide 14: protection works from a calm start

- **Shows:** the stress-episode table (unhedged, always 1M, always 3M, proposed) and two cards: where it works and where it fails.
- **Sources:** `episode_returns.csv`.
- **Say:** "This is the honest picture. The timed hedge protects against one kind of event: a sudden shock from a calm market, like January 2020 or August 2024. In a slow grind, or when VIX has already risen before the drop, it isn't holding anything, as in 2022, late 2024 and April 2025. Always-on puts protect more often, but they cost more than they saved over this period."

Slide 15: what the result depends on

- **Shows:** Figure 8 (heatmaps of CAGR and drawdown over buy threshold × first trigger), two text lines on pricing robustness, and three cards: one robust, two fragile.
- **Sources:** `sensitivity_thresholds.csv`, `sensitivity_pricing.csv`, `sensitivity_rule_by_tenor.csv`, `ex_covid_performance`.
- **Say:** "What's robust: buying only when VIX is below 12 to 15 beat buying regardless of VIX in every cell, and under every skew and spread assumption. What's fragile: tenor, because 1 and 2-month puts expired before the March 2020 crash, and above all the dependence on one crash. From July 2020 the proposal returned 14.9% against 15.4% unhedged with the same worst fall. That half-percent a year is the real cost of the insurance in normal times."
- **Be ready for:** this is the slide examiners probe most. Saying the fragility out loud before they ask is the strongest move in the talk.

Slide 16: limitations

- **Shows:** four cards: modelled premiums, short history, synthetic expiries, costs left out.
- **Say:** "Four limitations. Premiums are modelled from VIX, and real 3-month puts in calm markets probably cost more. The sample has one true crash and the thresholds were set on it. Expiries are synthetic. And there are no dividends, taxes or brokerage. The first extension I'd make is a longer India VIX history back to 2008, and real NSE option prices."

Slide 17: conclusion

- **Shows:** four conclusion bullets and an orange recommendation card.
- **Sources:** all from earlier slides.
- **Say:** "To sum up: Nifty option buyers pay about 2 vol points of risk premium, about 4 just after shocks. India VIX is a good real-time gauge of fear and of option prices, not of direction. Event premiums build for weeks and collapse in a day. And for a long-term investor, a 3-month put bought only when VIX is below 13 and sold into the first spike costs about half a percent a year in calm times and roughly halved the worst drawdown in this sample. Treat it as cheap crash insurance, not a return engine. Thank you."

How build_deck.js makes the deck

The deck is not hand-drawn. `build_deck.js` (466 lines of JavaScript) builds it with **pptxgenjs**, a Node.js library that writes PowerPoint files. The charts on slides 4, 7, 12 and 13 are native PowerPoint charts with their data embedded, so you can right-click and edit

them. The other charts are the PNG figures from the analysis. You can edit the `.pptx` directly in PowerPoint; you only need the script if you want to regenerate the whole deck.

Running it

```
npm install pptxgenjs
APPLY_THEME=<path to apply_theme helper> node build_deck.js
fdrm_vix_presentation.pptx
```

Two things to change on your own computer:

1. **The figures path.** Line 7 sets `FIG = "/mnt/project-files/fdrm-vix/figures"`, the shared project folder. Change it to your local `figures` folder, for example `path.join(__dirname, "figures")`.
2. **The theme helper.** Line 5 loads `applyTheme` from a slide-building toolkit used in this project, which is not on npm. It writes the deck's colour scheme and fonts into the file's theme. Without it, delete line 5 and the `await applyTheme(OUT, THEME);` line near the end. The deck still builds, but every colour referenced through the theme (most text and accents) falls back to PowerPoint's default Office colours. To keep the look, replace `C.text2`, `C.accent1` and so on with the hex values from `THEME.colors`.

Block by block

Theme (lines 10–22). `THEME` names the fonts (Cambria for headings, Calibri for body) and twelve colours: dark navy `dk2` (1B2A41) for backgrounds and titles, orange `accent1` (E8772E) for fear and high VIX, blue `accent2` (2A78D6) for calm and the proposal, grey `accent3` for neutral. `ORANGE_TINT` and `BLUE_TINT` are pale card fills.

Presentation setup (lines 24–31).

```
const pres = new pptxgen();
pres.layout = "LAYOUT_WIDE"; // 13.333 x 7.5
const C = pres.SchemeColor;
const W = 13.333, M = 0.6, CW = W - 2 * M;
```

The slide is 13.333 × 7.5 inches (16:9). `C` refers to theme colour slots rather than fixed hex codes, so changing the theme recolours the whole deck. `M` is a 0.6-inch margin and `CW` the usable content width; every position on every slide is computed from these.

Slide masters (lines 34–58). A master is a template that slides are built on.

- `TITLE_DARK`: navy background with a large title and subtitle placeholder. Used for slide 1.

- `CONTENT` : white background, title placeholder at the top, footer text "FDRM · Nifty options and India VIX" and a slide number. Used for slides 2–16.
- `CLOSING_DARK` : navy background with a title and slide number. Used for slide 17.

Titles go into placeholders (`{ placeholder: "title" }`), so they appear correctly in PowerPoint's outline view and accessibility tools.

Helper functions (lines 61–108). Small functions that draw the repeated elements, so each slide is a few lines.

Helper	Draws
`card`	A rounded rectangle used as a background panel
`numCircle`	A coloured circle with a number in it (the step markers)
`headDesc`	A bold heading with a description under it, in one text box
`stat`	A card with a big number and a label (the stat cards)
`bullets`	Turns a list of strings into bulleted paragraphs
`img`	Places a PNG at a given width; height = width × ratio, where the ratio is the image's pixel height ÷ width, so it never stretches
`barChart`	A native bar or column chart with value labels, no axis or gridlines, one colour per bar
`tableRows`	A table with a navy header row, alternating row shading and a bold first column

The slides (lines 110–460). Each slide follows the same pattern:

```
pres.addSection({ title: "Volatility and India VIX" });          // section
in PowerPoint's sidebar
s = pres.addSlide({ masterName: "CONTENT", sectionTitle: "…" });
s.addText("VIX usually overstates future volatility", { placeholder: "title"
});
img(s, "fig2_vrp.png", M, 1.45, 7.3, 882 / 1426, "alt text");
barChart(s, ["Low", "Normal", "High"], [2.1, 1.4, 4.0], [HEX.accent2,
HEX.accent3, HEX.accent1], { … });
s.addText(bullets([ … ]), { x: 8.25, y: 4.55, w: 4.48, h: 2.3, … });
s.addNotes("The blue line is India VIX …");
```

Repeated cards are laid out with simple arithmetic. For the four stat cards on slide 2: card width `cw = (CW − 3 × 0.3) / 4` (four cards, three 0.3-inch gaps) and card i starts at `x = M + i × (cw + 0.3)`. Every image and shape also gets alt text or an `objectName`, which makes the deck readable by screen readers and easier to edit.

Writing the file (lines 462–466). An async block waits for `pres.writeFile` to finish, then runs `applyTheme` on the saved file.

The numbers are copied, not linked

Every number on the slides is typed into `build_deck.js` from `results.json` and the output CSVs. If you change an assumption and re-run the Python, the deck will not update by itself. You would need to change the numbers in these places: slide 2 stat cards, slide 3 regime shares, slide 4 chart and bullets, slide 7 charts and bullets, slides 8–9 text and table, slides 11–15 charts, cards and table, and the notes on most slides. A worthwhile improvement: have the script read `output/results.json` and the CSVs with `require` and `fs`, so the deck rebuilds from the latest run.

The PDF

`fdrm_vix_presentation.pdf` was made by converting the pptx with LibreOffice:

```
soffice --headless --convert-to pdf fdrm_vix_presentation.pptx
```

The deck uses Calibri and Cambria, which are Microsoft fonts. On Linux, LibreOffice substitutes the free Carlito and Caladea fonts, which have exactly the same letter widths, so text wraps the same as in PowerPoint. If you convert on a machine without them, lines may wrap differently. Exporting to PDF from PowerPoint itself avoids this.